Adatbázis-kezelő rendszerek II.

Kivonatos jegyzet és órai feladatok

Készítette: Vassányi István, 2018-2019.

Tartalom

1. Bevezetés: a Northwind adatbázis 2

2. Adatmodellezés és kereszttáblás lekérdezés 4

Mező-orientált és rekord-orientált modellezés 4

Kereszttáblás lekérdezések, PIVOT/UNPIVOT 6

3. Adatbázis-programozás T-SQL nyelven 11

Utasítás-kötegek 11

A T-SQL procedurális kiterjesztései, szerver oldali logika 12

T-SQL változók és vezérlés 13

Hibakezelés T-SQL-ben 16

Kurzorok 20

A kurzorok helyes használata 22

A kurzorok típusai 22

Analitikus függvények és ablakozás 25

4. Tranzakció-kezelés T-SQL-ben 28

Az atomicitás biztosítása 28

Az izoláció biztosítása 32

Tranzakciók és az OLTP adatbázisok szerkezete 36

5. Szerver oldali tárolt logika 39

Tárolt eljárások 39

A tárolt eljárások visszatérési értékei 39

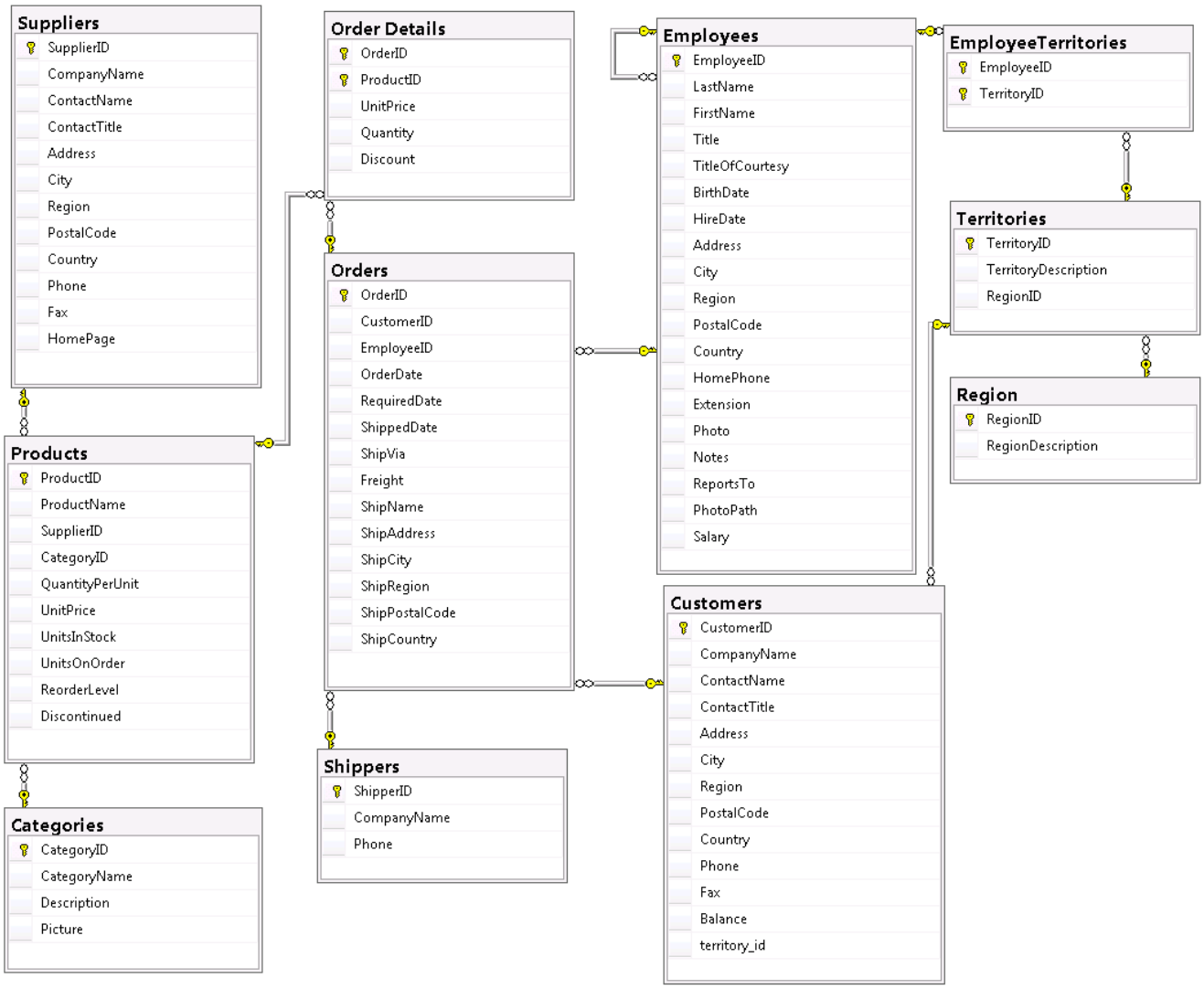
Beépített T-SQL függvények 43

Felhasználó által definiált függvények 43

Triggerek 45

1. Bevezetés: a Northwind adatbázis

A példák egy részéhez egy képzeletbeli kereskedőcég adatbázisát használjuk, az alábbi sémával.



Egy adatbázis lemásolásához használjuk a Management Studio Tasks -> Generate scripts parancsát, és az Advanced opciók között a Types of Data to script-nél válasszuk a Schema and data opciót. A generált scriptet futtassuk a saját adatbázisunkban. Így a kulcsok és kényszerek is átjönnek. A scriptek generálásához db_owner jogosultság szükséges a forrás-adatbázison.

A cég alkalmazottait az Employees tábla tartalmazza. A legtöbben ügynökök, akik egy bizonyos területért felelősek. Az egy ügynökhöz tartozó területeket az EmployeeTerritories tábla mutatja. Az ügynökök rendeléseket kötnek (Orders tábla) a cég ügyfeleivel (Customers tábla). A rendelésen egy vagy több rendelési tétel szerepel (Order Details tábla), egy tétel tartalma a megrendelt termék, a mennyisége, a rendeléskor érvényes egységár és a kedvezmény. A rendeléseket futárcégek szállítják ki, melyeket a Shippers tábla tartalmaz. A rendeléseket a cég raktárából szolgálják ki, melynek aktuális raktárkészletét a Products tábla tartalmazza. Minden terméket pontosan egy beszállítótól szereznek be (Suppliers tábla)

Az adatbázis érdekesebb mezői magyarul:

- **Categories**: termék-kategóriák

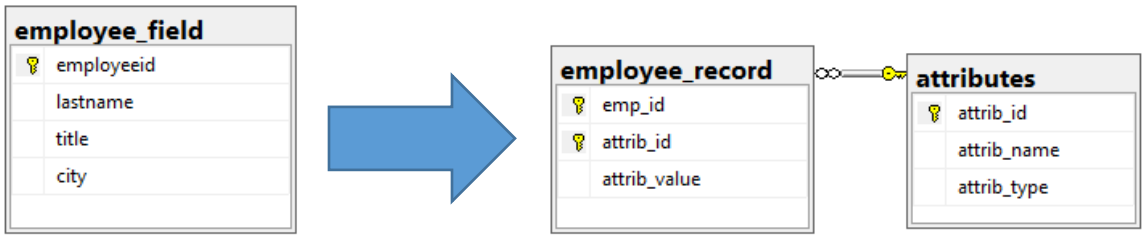
    - CategoryID: kategória-azonosító, kulcs
    - CategoryName: a kategória neve, például ’Beverages’ azaz italok
- **Products**: termékek
    - ProductID: termék-azonosító, kulcs
    - SupplierID: a beszállítóra mutató külső kulcs
    - CategoryID: a termék-kategóriára mutató külső kulcs (Categories tábla)
    - ProductName: a termék neve
    - UnitPrice: a termék éppen (mai napon) aktuális egységára (ajánlati ár)
    - UnitsInStock: a termék aktuális raktárkészlete
    - ReorderLevel: ha a raktárkészlet ez alá csökken, akkor a raktáros új rendelést küld a beszállítónak (Supplier)
    - UnitsOnOrder: a beszállítói rendelésen lévő (még be nem érkezett) egységek száma
    - Discontinued: ha 1, akkor a terméket már nem forgalmazzák, nem szerepelhet új rendelésen
- **Orders**: rendelések
    - OrderID: a rendelés azonosítója, kulcs
    - OrderDate: a rendelés dátuma
    - EmployeeID: az alkalmazottra mutató külső kulcs
    - CustomerID: a megrendelőre mutató külső kulcs (Customers tábla)
    - ShipVia: a rendelés kiszállítójára mutató külső kulcs (Shippers tábla)
- **Order Details**: a rendelés tételei
    - OrderID, ProductID: két mezős kulcs, illetve külön-külön külső kulcsok a rendelés (Orders) illetve a termék (Products) táblára
    - Unitprice: a tételen szereplő termékért a rendelés időpontjában fizetendő egységár (a Products táblában már lehet, hogy más van!)
    - Quantity: darabszám
    - Discount: árengedmény. Ha például az értéke 0.05, akkor a tétel értéke (UnitPrice*Quantity) 5%-kal csökkentendő.
- **Employees**: alkalmazottak
    - EmployeeID: alkalmazott-azonosító, kulcs
    - Lastname: vezetéknév
    - Title: beosztás, például „Sales Manager”
    - Country, City: az illető címe
    - ReportsTo: ugyanezen tábla EmployeeID mezőjére mutató külső kulcs: az illető főnöke
    - Emp_categ_id: a beosztás alapján a beosztás azonosítója
    - Salary: az alkalmazott fizetése
- **EmployeeTerritories**: alkalmazottak működési területei
    - EmployeeID: alkalmazottra mutató külső kulcs
    - TerritoryID: területre mutató külső kulcs

2. Adatmodellezés és kereszttáblás lekérdezés

Mező-orientált és rekord-orientált modellezés

Az eddig tanult adatmodellezés egyik alapelve az volt, hogy az objektumtípusok minden attribútumából a relációs modellben egy-egy mező lesz. Ez kétségtelenül egyszerűen programozható, azonban nagyon merevvé teszi az adatmodellt a későbbi változtatásokkal szemben. A kész információs rendszerben a logikai adatmodell változtatása a ráépülő és tőle függő logika miatt nagyon költséges és veszélyes. Tehát ha a problématér egy részében az attribútum-készlet változásai várhatók, és szeretnénk időtálló megoldást készíteni, akkor célszerű az adatmodell illető részében áttérni a mező-orientált modellről a rekord-orientált modellre.

A logikai adatbázis-modell átalakítását az alábbi ábra szemlélteti, egy három attribútumot tartalmazó objektumtípus (alkalmazott és a neve, beosztása, városa) esetén:



A táblák adattartalma pedig így változik:

employeeid	lastname	title	city
9	Dodsworth	Sales Repre...	London
11	Lamer	*NULL*	*NULL*

emp_id	attrib_id	attrib_value
9	1	Dodsworth
9	2	Sales Represent...
9	3	London
11	1	Lamer

attrib_id	attrib_name	attrib_type
1	Last name	text
2	Title	text
3	City	text

Az attribútum-készlet változásai egyszerűen menedzselhetők a pat_property törzstáblában az adatmodell változtatása nélkül. Mivel minden egyes korábbi mező-érték egy új rekordban lesz tárolva, ezért hívjuk ezt **rekord-orientált** modellnek, szemben a **mező-orientált** modellel.

Előnyök és hátrányok áttekintése:

	Flexibilitás	**Programozhatóság**	**Hatékonyság**
Mező-orientált modell	Merev. Egy új attribútum felvételéhez módosítani kell az adatmodellt és a ráépülő logikát.	Könnyen, gyorsan programozható. Minden GUI fejlesztő és ORM környezet automatizált megoldást kínál a mezők objektum-attribútumokhoz illetve GUI elemekhez (például szövegdobozokhoz) kötéséhez, sokszor programozás nélkül.	Üres (NULL) mezők

Rekord-orientált modell	Flexibilis, időtálló. Új attribútum felvételéhez elég egy új rekordot felvenni az attribútum-törzstáblába.	Nehézkes. Az egyetlen objektumpéldányhoz tartozó attribútumokat több rekordból kell összeszedni. Illesztő réteg, vagy külön GUI programozás szükséges.	Jobb hatékonyság: a nem ismert attribútum-értékeket nem tároljuk.

A különböző adatforrásokból származó adathalmazok összefésülését **adatföderációnak** is hívják. Az adatföderáció egy tipikus feladata a rekord-orientált modell átalakítása mező-orientált modellbe és fordítva.

PÉLDA: a példához a Northwind adatbázist használjuk. Most a cél az Employees tábla átalakítása rekord-orientált alakra.

```
use [saját ab]
select * from northwind.dbo.Employees
--drop table employee_field
select employeeid, lastname, title, city into employee_field from northwind.dbo.Employees
select * from employee_field
create table attributes (
       attrib_id int primary key,
       attrib_name nvarchar(100),
       attrib_type nvarchar(100))
go
insert attributes (attrib_id, attrib_name, attrib_type) values
       (1, 'Last name', 'text'), (2, 'Title', 'text'), (3, 'City', 'text')
--drop table employee_record
create table employee_record (
       emp_id int not null,
       attrib_id int not null references attributes (attrib_id),
       attrib_value nvarchar(500)
       primary key (emp_id, attrib_id))

--a feladat az employee_field tartalmának reprodukálása az employee_record táblában
--egy rekordból több rekord lesz:
insert employee_record (emp_id, attrib_id, attrib_value)
select employeeid, 1, lastname from employee_field where lastname is not null
union
select employeeid, 2, title from employee_field where title is not null
union
select employeeid, 3, city from employee_field where city is not null

--ellenőrzés: Lamer NULL rekordjai ugye nincsenek benne
select * from employee_record
--a kliens GUI számára fontosak a mezőnevek, típusok is:
select e.emp_id, a.attrib_name, a.attrib_type, e.attrib_value
from employee_record e inner join attributes a on e.attrib_id=a.attrib_id

--most alakítsuk vissza: több rekordból lesz egy rekord
select emp_id as employeeid,
min(case --MIN helyén lehetne MAX is, mert csak egy rekordot dolgoz fel
               when attrib_id=1 then attrib_value
               else null
       end)  as lastname,
min(case
               when attrib_id=2 then attrib_value
               else null
```

```
        end)  as title,
min(case
               when attrib_id=3 then attrib_value
               else null
        end)  as city
from employee_record group by emp_id
--megkaptuk az employee_field tartalmát
```

ÖNÁLLÓ FELADAT #1: Készítsük el és töltsük fel a Products tábla rekord-orientált változatát, ha a tárolt attribútumok a Productname, a Unitprice, és a Discontinued!

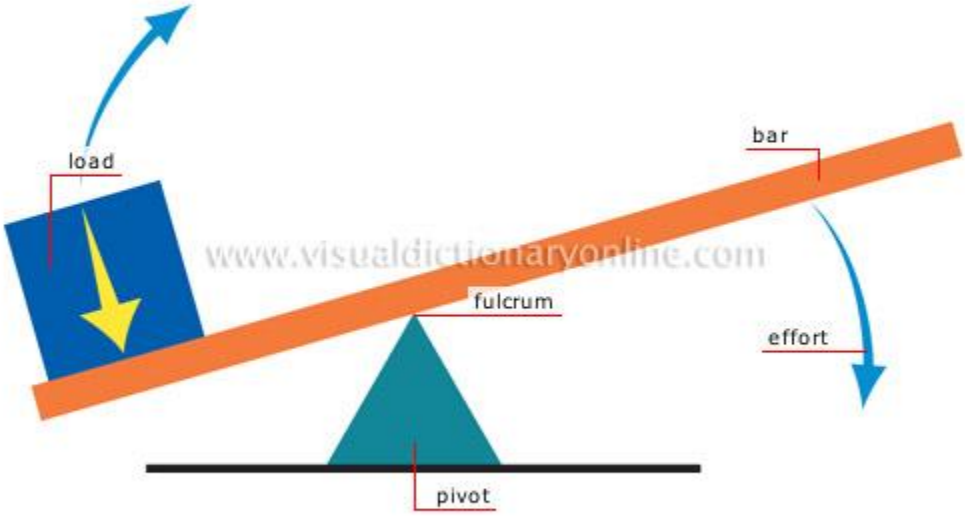
Megjegyzendő, hogy ha az eredeti táblára más táblák is hivatkoznak (ahogy például az Employees táblára az Orders tábla), akkor a mező -> rekord orientált átalakítás után meg kell tartanunk egy törzstáblát akár az attribútumok nélkül is.

Rekord-mező átalakítást a PIVOT/UNPIVOT SQL utasítással is lehet végezni, ha a mező-orientált formában a mezők egyetlen attribútum különböző értékeinek felelnek meg, vagy legalább az adattípusuk azonos.

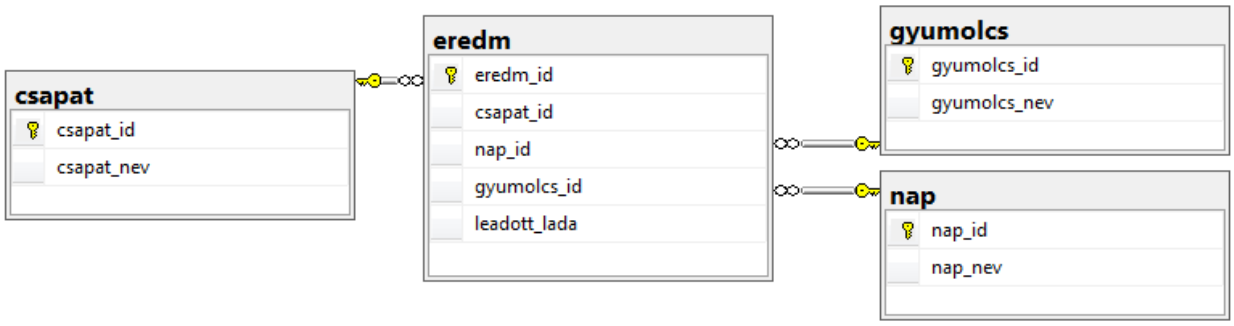
Kereszttáblás lekérdezések, PIVOT/UNPIVOT

Tegyük fel, hogy egy numerikusan mérhető eseménnyel, jelenséggel kapcsolatban különféle körülményeket (dimenziókat) tartunk nyilván. Például tároljuk, hogy melyik szüretelő csapat, melyik gyümölcsből, melyik napon (… stb.) mennyit szedett. Ekkor a numerikus eredmény tárolható rekord- vagy többféle mező-orientált formában is, és az ezek közötti átalakításra az SQL a PIVOT/UNPIVOT műveletet biztosítja.

Az angol „pivot” szó eredeti jelentése: billenési sarokpont, átbillentés. Jelen esetben egy dimenziót billentünk át a sorokból az oszlopokba.



PÉLDA: keresztáblás lekérdezések gyümölcsszedés közben, rekord-orientált formában:




```
drop table csapat
CREATE TABLE [dbo].[csapat] (
    [csapat_id] [int] NOT NULL primary key,
    [csapat_nev] [nvarchar] (50) NOT NULL
)
drop table gyumolcs
CREATE TABLE [dbo].[gyumolcs] (
    [gyumolcs_id] [int] NOT NULL primary key,
    [gyumolcs_nev] [nvarchar] (50) NOT NULL
)
drop table nap
CREATE TABLE [dbo].[nap] (
    [nap_id] [int] NOT NULL primary key,
    [nap_nev] [nvarchar] (50) NOT NULL
)
go
drop table eredm
CREATE TABLE [dbo].[eredm] (
       eredm_id int identity (1,1) primary key,
    [csapat_id] [int] NOT NULL references csapat (csapat_id),
    [nap_id] [int] NOT NULL  references nap (nap_id),
       gyumolcs_id int not null  references gyumolcs (gyumolcs_id),
    [leadott_lada] [int] NOT NULL
)
insert csapat (csapat_id, csapat_nev) values (1, 'Szorgos'),(2, 'Lusta')
insert gyumolcs (gyumolcs_id, gyumolcs_nev) values (1, 'alma'),(2, 'szilva')
insert nap (nap_id, nap_nev) values (1, 'hétfő'),(2, 'kedd'),(3, 'szerda')
insert eredm (csapat_id, nap_id, gyumolcs_id, leadott_lada) values
       (1,1,1,50), (1,2,1,60), (1,3,1,70), (1,1,2,100), (1,2,2,120), (1,3,2,140),
       (2,1,1,5), (2,2,1,6), (2,3,1,7), (2,1,2,10), (2,2,2,12), (2,3,2,14)
select * from eredm
--NÉHÁNY PÉLDA CSOPORTOSÍTÓ LEKÉRDEZÉSEKRE
--kérdés pl. a csapatok teljesítménye naponta
select cs.csapat_nev, n.nap_nev, sum(leadott_lada) as teljesítmény
from eredm e inner join csapat cs on cs.csapat_id=e.csapat_id
inner join nap n on n.nap_id=e.nap_id
inner join gyumolcs gy on gy.gyumolcs_id=e.gyumolcs_id --elhagyható
group by cs.csapat_id, cs.csapat_nev, n.nap_id, n.nap_nev
order by cs.csapat_nev, n.nap_nev

--csapatok teljesítménye gyümölcsönként
select cs.csapat_nev, gy.gyumolcs_nev, sum(leadott_lada) as teljesítmény
from eredm e inner join csapat cs on cs.csapat_id=e.csapat_id
inner join nap n on n.nap_id=e.nap_id --elhagyható
```

```
inner join gyumolcs gy on gy.gyumolcs_id=e.gyumolcs_id
group by cs.csapat_id, cs.csapat_nev, gy.gyumolcs_id, gy.gyumolcs_nev
order by cs.csapat_nev, gy.gyumolcs_nev

--ládaszám gyümölcsönként
select gy.gyumolcs_nev, sum(leadott_lada) as teljesítmény
from eredm e
inner join csapat cs on cs.csapat_id=e.csapat_id --elhagyható
inner join nap n on n.nap_id=e.nap_id --elhagyható
inner join gyumolcs gy on gy.gyumolcs_id=e.gyumolcs_id
group by gy.gyumolcs_id, gy.gyumolcs_nev
order by gy.gyumolcs_nev

--Az egyszerűség kedvéért legyen egyetlen szöveges kiinduló táblánk:
drop table eredm_pivot
select cs.csapat_nev, n.nap_nev, gy.gyumolcs_nev, e.leadott_lada
into eredm_pivot
from eredm e
inner join csapat cs on cs.csapat_id=e.csapat_id
inner join nap n on n.nap_id=e.nap_id
inner join gyumolcs gy on gy.gyumolcs_id=e.gyumolcs_id
go
```

Most a PIVOT utasítást használjuk arra, hogy a gyümölcsök az oszlopokban jelenjenek meg. A részei:

- A forrástábla megadása, gyakran egy beágyazott SQL SELECT formájában. Ha például *csak* a csapatok és a gyümölcsök érdekesek, a napok nem, akkor a csapatok és a gyümölcsök szerint kell csoportosítani.
- A PIVOT kulcsszó után az értékeket tartalmazó mező megnevezése, valmilyen aggregáló művelettel. Ne feledjük, ha ténylegesen nem történik aggregálás, csak átrendezés, mint a mi esetünkben is, akkor a SUM műveletet egyetlen értéken végezzük el, ezért egyszerűen hatástalan. Azonban a PIVOT szintaktikája előírja.
- A FOR kulcsszó után megadjuk a forrástábla azon mezőjét, amelynek különböző értékeit az oszlopokba szeretnénk tenni, illetve IN után a lehetséges értékek felsorolását. Ezekből az értékekből lesznek az új mezők.

Íme az eredmény:

```
select csapat_nev, nap_nev, pt.alma, pt.szilva
from eredm_pivot
       pivot (sum(leadott_lada) for gyumolcs_nev in (alma, szilva)) as pt
              --a SUM azért kell, mert lehetnének egyező sorok
order by csapat_nev, nap_nev

/*
ÖNÁLLÓ FELADAT:
•      Legyenek sorokban a napok és gyümölcsök, oszlopokban a csapatok */

select * from eredm_pivot where csapat_nev='lusta'

--most összesítünk: a napok nem érdekesek
--csapatok teljesítménye gyümölcsönként: sorokban a csapatok, oszlopokban a gyümölcsök
select pt.csapat_nev, pt.alma, pt.szilva
from (select csapat_nev, gyumolcs_nev, sum(leadott_lada) as leadott_lada
```

```
              from eredm_pivot group by csapat_nev, gyumolcs_nev) as forras --az alias
szintaktikai okból kell
       pivot (sum(leadott_lada) for gyumolcs_nev in ([alma], [szilva])) as pt

--sorokban a gyümölcsök, oszlopokban a csapatok
select gyumolcs_nev, pt.Lusta, pt.Szorgos
from (select csapat_nev, gyumolcs_nev, sum(leadott_lada) as leadott_lada
              from eredm_pivot group by csapat_nev, gyumolcs_nev) as forras
       pivot (sum(leadott_lada) for csapat_nev in ([Lusta], [Szorgos])) as pt

--mi lenne, ha kihagynánk a keddet? -> a belső lekérdezés változik
select gyumolcs_nev, pt.Lusta, pt.Szorgos
from (select csapat_nev, gyumolcs_nev, sum(leadott_lada) as leadott_lada
              from eredm_pivot
              where nap_nev<>'kedd'
              group by csapat_nev, gyumolcs_nev) as forras
       pivot (sum(leadott_lada) for csapat_nev in ([Lusta], [Szorgos])) as pt

/*
ÖNÁLLÓ FELADAT:
•      Legyenek sorokban a napok, oszlopokban a csapatok
•      ...és hagyjuk ki a szerdát
*/

--a sorokban lehet több szintű bontás is: sorokban a csapatok és napok, oszlopokban a
gyümölcsök
select csapat_nev, nap_nev, pt.alma, pt.szilva
from (select csapat_nev, nap_nev, gyumolcs_nev, sum(leadott_lada) as leadott_lada
              from eredm_pivot group by csapat_nev, nap_nev, gyumolcs_nev) as forras --
ehhez már nem is kellett volna belső lekérdezés
       pivot (sum(leadott_lada) for gyumolcs_nev in ([alma], [szilva])) as pt
order by csapat_nev, nap_nev
```

ÖNÁLLÓ FELADAT #2:

- Lásd a fenti példa-scriptben, beágyazva

Az UNPIVOT értelemszerűen visszacsinálja a PIVOT-ot, tehát a különböző mezőkből kivett értékekből egyetlen, megadott nevű mezőt készít. Az adattípusoknak egyezniük kell.

```
--induljunk ki egy pivotált táblából:
select csapat_nev, pt.alma, pt.szilva
into #temp
from (select csapat_nev, gyumolcs_nev, sum(leadott_lada) as leadott_lada
              from eredm_pivot group by csapat_nev, gyumolcs_nev) as forras
       pivot (sum(leadott_lada) for gyumolcs_nev in ([alma], [szilva])) as pt

select * from #temp

--és tegyük vissza gyümölcsöket a sorokba:
select * from #temp
select csapat_nev, gyumolcs_nev, leadott_lada
from #temp
       unpivot (leadott_lada for gyumolcs_nev in (alma, szilva)) as upt
--persze a csoportosítást már nem tudjuk visszacsinálni, ott információ veszett el
```

```
--ha nem volt csoportosítás, akkor a tábla teljesen visszaalakítható:
drop table #temp
go
select csapat_nev, nap_nev, pt.alma, pt.szilva
into #temp
from eredm_pivot
       pivot (sum(leadott_lada) for gyumolcs_nev in ([alma], [szilva])) as pt
order by csapat_nev, nap_nev

select csapat_nev, nap_nev, gyumolcs_nev, leadott_lada
from #temp
       unpivot (leadott_lada for gyumolcs_nev in (alma, szilva)) as upt
order by csapat_nev, nap_nev, gyumolcs_nev
```

ÖNÁLLÓ FELADAT #3: visszaalakítás a "sorokban a napok és a gyümölcsök, oszlopokban a csapatok" alakból

ÖNÁLLÓ FELADAT #3a (haladóknak): készítsünk egy kimutatást PIVOT segítségével a Products, Categories és Suppliers táblák alapján, melynek soraiban a beszállítók, az oszlopokban a termékkategóriák, az érték pedig az, hogy az illető szállítónak az adott kategóriában hány különböző terméke van. UNPIVOT-tal állítsuk vissza az eredeti lekérdezést.

Végül a teljesség kedvéért az Employees tábla átalakítása mező-orientált alakra a PIVOT használatával:

```
--a dolgozós rekordorientált modell visszaírása mezőorientáltra PIVOT segítségével
--Mi a mezőnév-készlet?
select distinct a.attrib_name
from employee_record e inner join attributes a on e.attrib_id=a.attrib_id
--eredmény:
[City]
[Last name]
[Title]
--ezzel a megoldás:
select emp_id, pt.[City], pt.[Last name], pt.[Title]
from (
       select e.emp_id, a.attrib_name, e.attrib_value
       from employee_record e inner join attributes a on e.attrib_id=a.attrib_id) as forras
pivot (max(attrib_value) for attrib_name in ([City],[Last name],[Title])) as pt
--max helyett lehetne min is
```

Ha az oszlopokba szánt mező értékkészletét nem tudjuk, vagy nem akarjuk felsorolni, akkor úgynevezett dinamikus PIVOT/UNPIVOT használható. Ebben az esetben a végrehajtandó PIVOT utasítás szövegét lekérdezésekkel, programozottan állítjuk elő és az EXECUTE paranccsal hajtatjuk végre.

3. Adatbázis-programozás T-SQL nyelven

Utasítás-kötegek

A T-SQL script SQL DDL és DML utasítások sorozata, ilyen lehet például egy adatbázis dump[1] is. Ezeket GO utasításokkal kisebb egységekre lehet bontani, egy ilyen egység a **köteg** (**batch**). A GO külön sorba írandó, nem SQL kulcsszó, csak a konzol programok (például SSMS) ismerik. A kötegenkénti feldolgozás lépései: szintaktikai ellenőrzés (parse), fordítás (compile), végrehajtás (execute). A köteg jellemzői:

- A T-SQL változók (lásd később) csak a kötegen belül érvényesek
- Mivel a szintaktikai ellenőrzés megelőzi a fordítást, szintaktikai hiba esetén a kötegből semmi sem hajtódik végre,
- Nem szintaktikai hiba esetén, például ha nem létezik a SELECT által hivatkozott tábla, a végrehajtás a következő köteg elejétől folytatódik, *kivéve* néhány speciális hibát, melyek esetén a végrehajtás a kötegen belül a következő utasítással folytatódik. Ilyen speciális hiba a nullával osztás és a kényszerek megsértése.

PÉLDA: utasításkötegek

```
--drop table pelda
create table pelda (szam int)
insert pelda values (44)
go
--1. kísérlet
update pelda set szam=21
go
selec * from pelda --szintaktikai hiba
go
select * from pelda --eredmeny 21, mert az első batch sikerult
go
--2. kísérlet: ugyanez egy kötegben
update pelda set szam=22
selec * from pelda --szintaktikai hiba
update pelda set szam=23
go
select * from pelda --eredmény: 21
--mert szintaktikai hiba esetén az egész batchból semmi sem hajtódik végre
--3. kísérlet
--ha az objektumnév hibás (nem szintaktikai hiba), akkor a hiba előtti rész végrehajtódik
go
update pelda set szam=22
select * from plda --Invalid object name
update pelda set szam=23 --ez már nem hajtódik végre
go
select * from pelda --eredmény: 22
go
--4. kísérlet: speciális hibák (nullával osztás, külső kulcs kényszer)
update pelda set szam=23
select 1/0 --division by zero
```

[1] A **dump** az adatbázis összes objektumát, vagy kiválasztott objektumait létrehozó és feltöltő SQL nyelvű (szöveges) script. Nem keverendő a **backuppal**, amely az adatbázist eredeti bináris formában tartalmazza. A dump tipikusan jóval nagyobb méretű, azonban technológiák és verziók között hordozhatóbb, mint a backup.

```
update pelda set szam=24 --végrehajtódik
go
select * from pelda --eredmény: 24
go
update pelda set szam=25
delete employees --The DELETE statement conflicted with the REFERENCE constraint
update pelda set szam=26 --végrehajtódik
go
select * from pelda --eredmény: 26
```

A T-SQL procedurális kiterjesztései, szerver oldali logika

Az SQL-nek számos procedurális kiterjesztése ismert. Például Oracle: PL/SQL, Postgres: PL/pgSQL, Microsoft: T-SQL. Ez utóbbi elemei a változók, vezérlő és egyéb utasítások.

A procedurális SQL-verziók létrehozásának egyik motivációja volt, hogy az **üzleti logikai réteg** legalább részben szerver oldali tárolt eljárások, függvények, triggerek stb. formájában legyen megvalósítható. Ennek **előnyei**:

- **Egyszerűség**: Ha az összes üzleti logika az adatbázisban van, akkor nem kell külön technológia az üzleti logikára, a vékony kliens csatlakozhat közvetlenül a szerverhez
- **Technológia-semlegesség**: ugyanazt a tárolt eljárást bármilyen technológiájú szoftver komponens vagy kliens program használhatja távoli eljárás-hívás formájában
- **Adatbiztonság**: A kliens programnak nem kell közvetlenül hozzáférnie az adatbázishoz, elég, ha a megfelelő tárolt eljárásokhoz van futtatási joga. Ha viszont a kliens program futtatja a logikát, tipikusan minden táblára írási-olvasási jogra van szüksége. A kliens program hibája vagy feltörése esetén ennek nagy ára lehet.
- **Menedzselhetőség**: A webes rendszerekhez hasonlóan a logika változtatható a vékony kliensek változtatása nélkül (ez különösen vállalati környezetben előnyös)
- **Hatékonyság**: Ha a logika *nagy tömegű* adattal dolgozik, lényegesen hatékonyabban (gyorsabban) fog futni, mint ha minden rekordot elküldünk egy kliensre és egy magas szintű objektum-orientált programnyelvben dolgozzuk fel. A hálózati forgalom kisebb, és az adatfeldolgozás erre optimalizált hardver-szoftver környezetben történik.
- **Átláthatóság**: Adat-közeli műveletek esetén az SQL-re (mely maga is specializált adatkezelő nyelv) épülő szintaktika sokkal egyszerűbb és átláthatóbb kódot eredményez, mint más magas szintű programnyelveken. Különösen áttekinthetetlen a dinamikus SQL használata bonyolult SQL lekérdezések setén.

Hátrányai:

- **Alacsony szint**: Általános programozási feladatokra az SQL nehezen és esetlenül használható
- **Szoftver-technológia**: Az adatbázis-oldali programozás szoftver-technológiai támogatása általában nehézkesebb (debug, verziókezelés, csapatmunka stb.)
- **Skálázhatóság**: Az adatbázis-szerverek skálázása rutinszerűen megoldható, de nagyon költséges. Az alkalmazás-szerverek skálázása olcsóbb.

Azok a hátrányok, hogy specializált szakértelem szükséges, illetve, hogy a technológiák közti váltás költséges („vendor lock-in”), egyformán jellemzőek az adatbázis-oldali illetve alkalmazás-szerveres architektúrára, ezért nem szerepelnek a hátrányok között.

A gyakorlatban, egy konkrét alkalmazásnál a funkcionalitás választja el az alkalmazás-szerverre (vagy a kliensbe) építendő logikát az adatbázis oldali logikától: az adat-közeli funkcionalitást, különösen nagy tömegű és jól strukturált adat feldolgozása esetén célszerű az adatbázis-szerveren megvalósítani. Az adatbázis create dump-ja (a tárolt eljárásokkal együtt) a fejlesztési projektben a menedzselt forráskód részét kell, hogy képezze.

T-SQL változók és vezérlés

A MS SQL Server által támogatott T-SQL procedurális kiterjesztése a következő kulcsszavakat használja:

- DECLARE: változók deklarálása. A változók neve @-tel kezdődik. Deklaráláskor értéket is lehet adni. *Az első értékadásig a változó értéke NULL.* A változó csak a kötegen belül látható.
- SET: explicit értékadás változónak
- BEGIN…END: egynél több utasítást tartalmazó utasításblokk
- IF/ELSE: feltételes elágazás
- WHILE/BREAK/CONTINUE: elöl tesztelő ciklus. Másféle ciklus nincs.
- RETURN: feltétel nélküli kilépés kötegből, utasításblokkból vagy tárolt eljárásból
- TRY/CATCH/THROW/RAISERROR: kivételek kezelése
- WAITFOR/DELAY/TIME: várakozás időtartamig vagy időpontig
- GOTO <címke>

PÉLDA: ciklus és változók használata, SQL nélkül

```
declare @i int, @eredm int
set @i = 1
--ehelyett működne ez is: declare @i int = 1, @eredm int = 0
set @eredm=0
--azonos eredményt ad: select @eredm=0
while @i < 50 begin
    set @eredm = @eredm + @i
    set @i = @i + 1
end
print 'az 50-nél kisebb számok összege: '+cast(@eredm as varchar(15))
go
--print @eredm --Must declare the scalar variable "@eredm".
--a változó csak a kötegen belül látható
```

ÖNÁLLÓ FELADAT #4: írjunk egy scriptet, amely kiírja az 1000-nél kisebb Fibonacci-számokat! Segítség: az első 5 szám az 1, 1, 2, 3, 5

PÉLDA: változók használata SQL-be ágyazva + feltételes elágazás. A keresett nevű dolgozó címének kiírása.

```
use northwind
--select * from Employees
declare @name nvarchar(20), @address nvarchar(max)
set @name='Fuller%'
select @address=Country+', '+City+' '+Address
       from Employees where LastName like @name
```

```
if @address is not null print 'A talált cím: ' + @address
/*megjegyzés: az utolsó sor működik. de jobban olvasható így:
if (@address is not null) begin
	print 'A talált cím:' + @address
end
*/
go
```

ÖNÁLLÓ FELADAT #5: írjunk egy scriptet, amely a kapott nevű termék termék-kategóriáját visszaírja

Figyelem: ha a fenti példát a 'King' paraméterrel futtatjuk, furcsa eredményt kapunk, ugyanis 2 King van a táblában. A SELECT @változó=[mezőnév vagy kifejezés] FROM… WHERE… szerkezetben:

- Ha egy illeszkedő rekordot sem találunk, akkor nem történik értékadás a változónak, tehát az értéke továbbra is NULL lesz (ha deklarálása óta nem adtunk neki értéket). Ezt fent ellenőriztük is. ***Kivétel***: ha a változóba egy COUNT() eredményét íratjuk, akkor üres recordset esetén is történik értékadás, a változó értéke 0 lesz[2].
- Ha egynél több illeszkedő rekordot találunk, a változó végső értéke az UTOLSÓ rekordból fog származni. Hibaüzenetet nem kapunk, ezáltal komoly logikai hiba is előfordulhat, ha nem kulcs szerint keresünk rekordokat. Hogy ezt elkerüljük, és mivel más procedurális SQL változatok (például PL/SQL) ilyen esetben futásidejű hibát adnak, célszerű mindig kulcs szerint keresni, *vagy* ha a kulcs nem ismert, mint ebben a példában, akkor:
  - Az értékadás előtt, vagy azzal egy időben ellenőrizni a találatok számát, és egynél több találat esetén hibát generálni
  - Ha nem gond a több találat, és tényleg az utolsó rekord kell, akkor MINDIG expliciten **rendezni kell** a rekordhalmazt, ugyanis a rendezés nélküli SELECT rekord-sorrendje nem determinisztikus.
- Ha a változó mindkét oldalon szerepel, akkor alkalmas arra is, hogy több visszaadott rekord minden értékét tárolja (lásd az alábbi utáni példát).

PÉLDA: az előző script javítva a találati szám ellenőrzésével

```
declare @name nvarchar(20), @address nvarchar(max), @res_no int
set @name='Kinkg%'
select @res_no=count(*) from Employees where LastName like @name
--Figyelem, ebben az esetben üres recordsetre a @res_no értéke nem NULL, hanem 0 lesz!
if @res_no=0 print 'Nincs találat.'
else if @res_no>1 print 'Több, mint 1 találat.'
else begin
	select @address=Country+', '+City+' '+Address from Employees where LastName like @name
	print 'A talált cím: ' + @address
end
go
--megjegyzés: mivel csak 1 rekordot várunk, a 2. select megúszható lett volna így:
declare @name nvarchar(20), @address nvarchar(max), @res_no int
set @name='Buchanan%'
select @res_no=count(*), @address=max(Country+', '+City+' '+Address)
	from Employees where LastName like @name
if @res_no=0 print 'Nincs találat.'
else if @res_no>1 print 'Több, mint 1 találat.'
```

[2] A kivétel kivétele: ha az üres halmazt adó lekérdezés csoportosítva volt, nem történik értékadás, például: `select @i=count(*) from Products where 1=0 group by ProductID --@i értéke NULL marad`

```
else print 'A talált cím: ' + @address
go
```

ÖNÁLLÓ FELADAT #6: egészítsük ki az előző (termék-kategóriás) scriptet a találati szám ellenőrzésével

PÉLDA: Írassuk ki az összes alkalmazott nevét egyetlen listában!

```
declare @nev_lista nvarchar(max) = ''
select @nev_lista = @nev_lista + ', '+ LastName from Employees order by Lastname
select 'A nevek listája: ' +  right(@nev_lista, len(@nev_lista)-1) lista

--SQL Server 2017-ben vagy afelett így is megoldható:
select string_agg(lastname, ', ') within group (order by Lastname) lista from Employees
```

A fenti példában a kiíratás előtt levágtuk a felesleges vesszőt az eredmény-string elejéről.

Egy T-SQL-ben lévő SELECT *vagy* változó(k)nak ad értéket, *vagy* a hívó kliensnek ad vissza egy rekordhalmazt, a kettőt keverni nem lehet. Az alábbi script hibára fut:

```
declare @lastname nvarchar(20), @emp_id int = 1
select @lastname = LastName, Address from Employees where EmployeeID = @emp_id
print @lastname

"Msg 141, Level 15, State 1, Line 108
A SELECT statement that assigns a value to a variable must not be combined with data-retrieval
operations."
```

A változók egyéb DML utasításokba is ugyanúgy beépülnek, mint a SELECT-be.

PÉLDA: Emeljük meg a megtalált dolgozó fizetését 10%-kal! A saját adatbázisunkban dolgozzunk!

```
set nocount on
declare @name nvarchar(20), @address nvarchar(max), @res_no int, @emp_id int
set @name='Fuller%'
select @res_no=count(*) from Employees where LastName like @name
if @res_no=0 print 'Nincs találat.'
else if @res_no>1 print 'Több, mint 1 találat.'
else begin  --épp egy találat
       select @address=Country+', '+City+' '+Address, @emp_id=EmployeeID
              from Employees where LastName like @name
       print 'Dolgozó ID: ' + cast(@emp_id as varchar(10)) + ', cím: ' + @address
       update Employees set salary=1.1*salary where EmployeeID=@emp_id
       print 'Fizetés növelve.'
end
go
```

ÖNÁLLÓ FELADAT #7: Kezeljük azt az esetet, mikor beérkezett egy rendelés a raktárba a beszállítótól! Ehhez a korábbiak alapján írjunk egy scriptet, amely a kapott nevű termék (csak egyetlen termék!) UnitsInStock mezőjének értékét növeli a UnitsOnOrder mező értékével, a UnitsOnOrder mezőt pedig 0-ra állítja.

Feladat- kiegészítések gyorsan haladóknak:

- írjuk vissza az új raktárkészletet!
- ha a talált terméknek nem is volt függő rendelése, tehát UnitsOnOrder=0, akkor adjunk figyelmeztetést, és ne csináljunk semmit!

ÖNÁLLÓ FELADAT #8: írjunk egy scriptet, amely adott sorszámú rendelés tételei alapján aktualizálja a Products.UnitsInStock mezőjének értékét! Tehát elégítsük ki a rendelést a raktárból. Egyelőre ne foglalkozzunk azzal az esettel, mikor a rendelés valamelyik tétel esetén meghaladja a raktárkészletet.

Hibakezelés T-SQL-ben

A T-SQL scriptben lévő DML és egyéb utasítások közben előfordulhatnak hibák. A hibára jellemző a hiba **szintje**, mely alapján a fő osztályok:

- 0-9: figyelmeztetések, például „Null value is eliminated by an aggregate or other SET operation" (ez kikapcsolható: SET ANSI_WARNINGS OFF)
- 10-16: olyan hibák, melyeket a felhasználó ki tud javítani. A legtöbb, rendszer által generált hibaüzenet 16-os szintű.
- 17-19: a felhasználó által nem javítható, de adatvesztést még nem okozó hibák, mint például a rendszergazda által beállított rendszerparaméter-határok túllépése. A kapcsolat nem szakad meg.
- 20-23: kritikus hibák, melyek egyes esetekben adatvesztést okozhatnak. A kapcsolat megszakad.
- 24: hardver hiba, például az adatbázis fájlok elvesztése

Bővebben: https://docs.microsoft.com/en-us/sql/relational-databases/errors-events/database-engine-error-severities

A hibát a **száma** azonosítja. Ezen kívül a szerver még egy hibaüzenetet is generál.

PÉLDA: check constraint megsértésére, 16-os szintű hiba:

```
update Products set UnitsInStock=-1 where ProductID=3 --check constraint sértés
Msg 547, Level 16, State 0, Line 360
The UPDATE statement conflicted with the CHECK constraint "CK_UnitsInStock". The conflict
occurred in database "northwind", table "dbo.Products", column 'UnitsInStock'.
The statement has been terminated.
```

Megjegyzés: a fenti üzenetben is szereplő State a hiba fajtáját (altípusát) azonosítja[3].

Figyelem: ha a hiba nincs kezelve (ezt lásd később), akkor a hiba a kliens oldali beállításoktól függően megjelenik a GUI-n, és ezzel komoly gondot okozhat. Például egy magyar nyelvű GUI-n egy angol nyelvű, a felhasználó számára teljesen érthetetlen hibaüzenet kifejezetten rossz benyomást kelt.

Az alapbeállításokkal a 10-es szint alatti hibaüzeneteket a kliensek eldobják, mert csak figyelmeztetéseknek tekintik őket.

PÉLDA egy konkrét hiba figyelésére a **@@error** csak olvasható globális T-SQL változó (igazából függvény) használatával:

```
go
select 1/0
if @@ERROR=8134 print '0-val osztottunk'
go
```

Ez a függvény csak az aktuális utasítást *közvetlenül* megelőző utasítás által kiváltott hiba számának lekérdezésére alkalmas.

[3] A hibakódok és state-ek listáját lásd: https://docs.microsoft.com/en-us/sql/relational-databases/errors-events/database-engine-events-and-errors?view=sql-server-ver15

Ha egy blokkban *bárhol* előforduló hibára szeretnénk felkészülni, akkor a TRY/CATCH blokkal kezelhető a kapott hiba. Hiba esetén a TRY blokkból a végrehajtás azonnal átadódik a CATCH blokknak, ahol a hiba minden paramétere lekérdezhető. Ha volt CATCH, akkor nem megy hibaüzenet a kliensnek (16-os szint alatt).

PÉLDA TRY/CATCH hibakezelésre:

```
create table #log(time_stamp datetime, err_num int)
go
begin try
    update Products set UnitsInStock=-13 where ProductID=3 --check constraint sértés
       select 1/0
end try
begin catch
    select
        ERROR_NUMBER() AS ErrorNumber,
        ERROR_SEVERITY() AS ErrorSeverity,
        ERROR_STATE() AS ErrorState,
        ERROR_PROCEDURE() AS ErrorProcedure,
        ERROR_LINE() AS ErrorLine,
        ERROR_MESSAGE() AS ErrorMessage
    insert #log (time_stamp, err_num) values (getdate(), ERROR_NUMBER())
end catch
select * from #log
go
```

A fenti ERROR_ függvények csak CATCH blokkban használhatók. A TRY/CATCH nem kezeli a 11-nél kisebb szintű hibákat, mivel ezek figyelmeztetések. A 19-nél magasabb szintűeket akkor kezeli, ha a hiba nem okozza a kapcsolat megszakadását.

ÖNÁLLÓ FELADAT #9: Az alábbi kis script a Products.UnitsInStock (raktárkészlet) értékét aktualizálja, mikor egy rendelést kiszolgálnak a raktárból.

```
select * from [Order Details] where OrderID=10248 --11: 42, 42: 40, 72:40
select ProductID, UnitsInStock from Products where ProductID in (11, 42, 72) --82, 20, 20
--az utolsó 2 tétel miatt az alábbi update hibára fut (nem lehet negatív raktárkészlet)
go
declare @order_id int = 10248
update Products set UnitsInStock=UnitsInStock-od.Quantity
from [Order Details] od inner join Products p on p.ProductID=od.ProductID
where od.OrderID=@order_id
print cast(@order_id as varchar(20))+' számú rendelés a raktárba átvezetve.'--hibás
üzenet
```

Egészítsük ki a fenti scriptet TRY/CATCH hibakezeléssel! Hiba akkor történik, ha a raktárkészlet 0 alá csökkenne. A megoldás lépései:

1. Tudjuk, hogy a UnitsInStock mezőn van egy CHECK constraint, ezért a script TRY részében megpróbáljuk elvégezni az UPDATE utasítást az érintett termékekre
2. A CATCH részben ellenőrizzük, hogy az 547-es hiba (check constraint megsértése) történt-e. Ha igen, ilyen jellegű hibaüzenetet adunk, mint például: „RAKTÁRKÉSZLET-HIBA: 10248 számú rendelésen a Queso Cabrales (11) tétel nem teljesíthető.”
3. Ha nem 547-es hiba történt, akkor általános hibaüzenetet generálunk, például: „EGYÉB HIBA: Divide by zero error encountered. (8134)” A szöveget ekkor az ERROR_MESSAGE() függvény szolgáltatja.

Természetesen az előre várható hibák kivédhetők előzetes ellenőrzésekkel is. A fenti feladatra még vissza fogunk térni a triggerek kapcsán.

PÉLDA egy egyszerű üzleti folyamat szerver oldali megvalósítására. Valaki egy egytételes telefonos rendelést rögzít az adatbázisba. A telefonon bemondják a vásárló azonosítóját (szöveges), a termék nevét és a mennyiséget. A products, customers, orders és order details táblákat kell módosítani. Az eredmény egy hibakód és egy üzenet.

```
--a szükséges táblák: products, customers, orders, order details
--ezekről készítsünk másolatot a saját adatbázisunkba!

--változók
declare @prod_name varchar(20), @quantity int, @cust_id nchar(5)
                      --a telefonból (a vevő-ID szöveges)
declare @status_message nvarchar(100),  @status int --a folyamat visszatérési értékei
declare @res_no int --találatok száma
declare @prod_id int, @order_id int --azonosítók
declare @stock int --raktárkészlet
declare @cust_balance money --a vevő egyenlege
declare @unitprice money --a termék egységára

-- paraméterek
set @prod_name = 'boston'
set @quantity = 10
set @cust_id = 'AROUT'

begin try
       select @res_no = count(*) from products where productname like '%' + @prod_name + '%'
       if @res_no <> 1 begin
              set @status = 1
              set @status_message = 'HIBA: a terméknév nem egyértelmű.';
       end else begin
              -- ha egy terméket találunk, megkeressük a kulcsot és a raktárkészletet
              select @prod_id = productID, @stock = unitsInStock from products where
productName like '%' + @prod_name + '%'
              -- van-e raktáron? a rendelt mennyiség elérheto-e?
              if @stock < @quantity begin
                     set @status = 2
                     set @status_message = 'HIBA: a raktárkészlet nem fedezi a megrendelést.'
              end else begin
              -- mennyibe kerül? Van elég pénze a vevönek?
                     select @cust_balance = balance from customers where customerid =
@cust_id
                            --ha nem találjuk a vásárlót, akkor az egyenleg null
                            --mivel kulcsra keresünk, max. 1 találat lehet
                     select @unitprice = unitPrice from products where productID = @prod_id
                            --nincs discount
                     if @cust_balance < @quantity*@unitprice or @cust_balance is null begin
                            set @status = 3
                            set @status_message = 'HIBA: a vásárló nem található vagy az
egyenlege túl alacsony.'
                     end else begin
              -- Ellenőrzések vége, elkezdjük a tranzakciót (2 lépés)
              -- 1. vásárló egyenlegének frissítése
                     update customers set balance = balance-(@quantity*@unitprice) where
customerid=@cust_id
              -- 2. új bejegyzés az Orders, Order Details táblákba
                            insert into orders (customerID, orderdate) values (@cust_id,
getdate()) --orderid: identity
                            set @order_id = @@identity  --az utolsó identity insert eredménye
```

```
                                insert [order details] (orderid, productid, quantity, UnitPrice)
                                               --itt a hiba
                                       values(@order_id, @prod_id, @quantity, @unitprice)
                                               --itt a hiba
--                              insert [order details] (orderid, productid, quantity, UnitPrice,
Discount) --ez a jó
--                                     values(@order_id, @prod_id, @quantity, @unitprice, 0) --ez
a jó

                                set @status = 0
                                set @status_message = cast(@order_id as varchar(20)) + ' sz.
rendelés sikeresen felvéve.'
                        end
                end
        end
        print @status
        print @status_message
end try
begin catch
        print 'EGYÉB HIBA: '+ ERROR_MESSAGE() + ' (' + cast(ERROR_NUMBER() as varchar(20)) +
')'
end catch
go

--teszteléshez beállítjuk a paramétereket
set nocount off
update products set unitsInStock = 900 where productid=40
update customers set balance=1000 where CustomerID='AROUT'
delete [Order Details] where OrderID in (select orderid from Orders where CustomerID='AROUT'
and EmployeeID is null)
delete Orders where CustomerID='AROUT' and EmployeeID is null
--most futtatjuk a scriptet, utána ellenőrzés:
select * from Customers where CustomerID='AROUT'
select * from Products where productid=40
select top 3 * from Orders where CustomerID='arout' order by OrderDate desc

--Tehát elvileg minden rendben. Azonban a discount mező NOT NULL kényszere miatt a futtatás
során hiba történik:
--"EGYÉB HIBA: Cannot insert the value NULL into column 'Discount'"
--DE! a vásárló egyenlegét azért sikerült csökkenteni!

--további hibalehetőség: ha egyszerre több konkurens kérés érkezik, akkor
--érdekes hibák léphetnek fel.

--javítás után teszteljük a másik két ágat is!
```

A példa tanulsága az, hogy még egy ilyen egyszerű, és a logikai hibákra felkészített script is tranzakcionális támogatást kíván a valóságban. A tranzakció-kezeléssel később foglalkozunk.

ÖNÁLLÓ FELADAT #10 (szorgalmi): Raktárkészlet-karbantartó script illetve tárolt eljárás. Az eljárást a raktáros használja arra, hogy egy létező termék raktárkészletét növelje, vagy egy új cikket vegyen fel a raktárba. Bemenő paraméterek (konkrét példákkal):

- set @termeknev = 'Raclette'
- set @mennyiseg = 12
- set @szallito = 'Formago Company'

Ekkor a script '%raclette%'-re illő terméknevet keres a Products táblában. Ha több mint 1 találat van, hibaüzenettel kilép. Ha pont 1 találat van, akkor annak a raktárkészletét (UnitsInStock) növeli 12-vel. Ha

nincs találat, akkor felvesz egy új Raclette nevű terméket, melynek a szállítóját (supplier) a 3. paraméter alapján próbálja beállítani, a fentihez hasonló logikával (több találat: hiba, 1 találat: megvan, 0 találat: új supplier rekord felvétele). Javasolt lépések:

1. lépés: olyan lekérdezés, amely visszaadja a mintára illeszkedő rekordok számát
2. lépés: beszúró INSERT illetve módosító UPDATE utasítás összerakása
3. lépés: a lekérdezések beágyazása egy IF feltételt használó script-be
4. lépés: a script tesztelése

Kurzorok

Kurzor alatt az adatbázis-technológiában **egy rekordhalmazt és egy ezen mozgó iterátort (rekordmutatót)** értenek. Minden SQL SELECT utasítás eredménye egy rekordhalmaz. A leggyakoribb esetben ezt a rekordhalmazt a szerver visszaküldi a kliensnek[4], amely a kliens oldalon, a kliens erőforrásaival (a memóriában) dolgozza azt fel, tipikusan végigiterál rajtuk. Ezt úgy nevezik, hogy a rekordhalmaz feldolgozásához **kliens oldali kurzort** használtunk. Ritkábban azonban előfordul, hogy az eredmény-rekordhalmaz túl nagy méretű, és a kliens erőforrásainak kímélése, illetve a hálózat lökésszerű terhelésének elkerülése érdekében a szerveren hagyjuk a rekordhalmazt, és a kliens olyan ütemben kapja meg a rekordokat a hálózaton keresztül, ahogy ténylegesen felhasználja őket. Ez folyamatos, de kis mértékű hálózati forgalmat eredményez, hasonlóan a video streaming elvéhez. Ez a **szerver oldali kurzor**.

A szerver illetve kliens oldali kurzor közötti helyes választás a fentiek szerint természetesen alkalmazás-függő. A programozó a felhasznált technológiával, könyvtárakkal és ezek beállításaival tudja a kurzor helyét befolyásolni. Például a C# (ADO.NET) esetén a DataReader osztály szerver oldali kurzort használ, ha pedig leválasztott kliens oldali kurzort szeretnénk, egy DataSet egyedet kell feltöltenünk egy Data Adapter segítségével.

PÉLDA kliens oldali kurzor használatára Php nyelven (nem része az anyagnak). A tipikus lépéseket az I-VII. római számok jelzik. A példa egyúttal bemutatja, hogyan lehet prepared statement használatával megakadályozni az SQL injektálásra[5] alapozott támadást.

```
<html><body>
DEMO: Kliens oldali kurzor használata <br>
<?php
//I. kapcsolódás a szerverhez
$mysqllink=mysqli_connect("[szerver neve]","[login név]","[jelszó]") or die("Hiba:
Kapcsolódási hiba.<br>");
//II. kapcsolódás az adatbázishoz
mysqli_select_db($mysqllink, "[adatbázis neve]") or die("Hiba: Nem létező adatbázis.<br>");
if ($stmt = mysqli_prepare($mysqllink, "SELECT title, isbn FROM books WHERE title LIKE ?")) {
        $keresett_cim = "%SQL%"; //felhasználótól származó paraméter
        mysqli_stmt_bind_param($stmt, "s", $keresett_cim);
//III. lekérdezés futtatása, találati szám ellenőrzése
        mysqli_stmt_execute($stmt);
```

[4] Kliens lehet az általunk is használt SQL Server Management Studio, egy alkalmazás-szerver, vagy egy dedikált kliens program.

[5] Az **SQL injektálás** során a felhasználói inputba rosszindulatú SQL kódot írnak arra számítva, hogy azt a szerver oldalon változtatás és ellenőrzés nélkül végrehajtják. Prepared statement használatakor az utasítás a felhasználótól származó paraméter (a példában a könyvcím egy részlete) hozzárendelésekor már nem változhat meg, ezért hiába van SQL kód a paraméterben, az nem fog lefutni.

```
        mysqli_stmt_store_result($stmt); //a kliens oldalra (webszerver memóriába) hozza az
eredményt
        if (mysqli_stmt_num_rows($stmt) == 0) { echo "Nincs találat! <br>";
        } else {
//IV. kliens oldali rekordhalmaz (kurzor) objektum létrehozása
                mysqli_stmt_bind_result($stmt, $cim, $isbn); //változók kurzorhoz kötése
//V. a kurzor iterálása
                echo "<table width=700 align=center border=1 cellspacing=1> <tr> <td
align=center> Cím </td> <td align=center> ISBN  </td></tr>";
                while (mysqli_stmt_fetch($stmt)) {
                        echo "<tr><td>".$cim."</td><td>".$isbn."</td></tr>";
                }
                echo "</table>";
        }
//VI. a kurzor felszabadítása
        mysqli_stmt_close($stmt);
}
//VII. a kapcsolat bontása
mysqli_close($mysqllink);
?>
</body></html>
```

Tárgyunk, a szerver oldali programozás szempontjából természetesen a szerver oldali kurzorok az érdekesek, ezért a továbbiak ezekre vonatkoznak. A T-SQL-ben explicit módon deklarálhatunk egy kurzort, amely a forrásául szolgáló, a szerver memóriájában lévő rekordhalmaz rekordjaihoz enged hozzáférést rekordonként, a leggyakoribb esetben a rekordhalmaz WHILE ciklussal való végigiterálása céljából. Ezáltal a procedurális programozás megszokott módszerei válnak használhatóvá az adatbázis-oldali programozásban.

T-SQL scriptekben a kurzorok vezérlése a DECLARE…CURSOR FOR…, OPEN, FETCH, CLOSE, DEALLOCATE utasításokkal lehetséges. Ezek használatát az alábbi példa mutatja.

PÉLDA

```
go
declare @emp_id int, @emp_name nvarchar(50), @i int, @address nvarchar(60)
declare cursor_emp cursor fast_forward for
    select employeeid, lastname, address from employees order by lastname
set @i=1
open cursor_emp
fetch next from cursor_emp into @emp_id, @emp_name, @address
while @@fetch_status = 0
begin
    print cast(@i as varchar(5)) + '. ügynök:'
    print concat('ID: ', @emp_id, ', Név: ', @emp_name, ', cím: ', @address)
    set @i=@i+1
    fetch next from cursor_emp into @emp_id, @emp_name, @address
end
close cursor_emp
deallocate cursor_emp
go
--ez egyenértékű egy ilyen SELECT utasítással:
select 'ID: ' + cast(employeeid as varchar(5)) + isnull(', Név: ' + lastname, '') + isnull( ',
cím: ' + address, '')
from employees order by lastname
--illetve ha az ügynök sorszámát is szeretnénk:
select cast(row_number() over(order by lastname) as varchar(50))+
'. ügynök: ID: ' + cast(employeeid as varchar(5)) + isnull(', Név: ' + lastname, '') + isnull(
', cím: ' + address, '')
from employees
```

```
--tehát ehhez még nem kellett volna kurzor…
```

A kurzorok egymásba is ágyazhatók, a deklarációtól kezdve a deallokálásig.

ÖNÁLLÓ FELADAT #11: készítsünk olyan scriptet, amely az amerikai vásárlók (customers tábla, country mező) rekordjain iterál és kiírja minden vásárló nevét, és hogy eddig hány rendelése volt

ÖNÁLLÓ FELADAT #12 (szorgalmi): az előző feladatot fejlesszük tovább úgy, hogy egy beágyazott kurzorral írassuk ki az illető vásárló rendeléseinek a dátumait is (leprogramozzuk az INNER JOIN műveletet)

A @@FETCH_STATUS globális változó 0 értékkel tér vissza, ha sikeres volt a rekordmutató által aktuálisan mutatott pozícióról a rekord beolvasása a változókba. Ha a mutató a rekordhalmaz végére ért, vagy a mutató által mutatott rekordot közben törölték, az érték nem 0. A FETCH után megadott változók sorrendjének és típusának követni kell a kurzor rekordképét. A kurzor által elfoglalt memória felszabadítása, tehát a forrás-rekordhalmaz törlése csak a DEALLOCATE hatására történik meg[6].

A kurzorok helyes használata

Célszerű kurzort használni, ha:

- A probléma a jellege miatt a hagyományos procedurális megközelítéssel, rekordonkénti feldolgozással sokkal könnyebben és átláthatóbban oldható meg, mint SQL DML segítségével, vagy nem is oldható meg máshogy, *és*
- A teljesítmény (válaszidő) a kis várható rekordszám vagy egyéb alkalmazás-specifikus körülmény miatt nem kritikus

Tipikus alkalmazási területek:

- Rekordok egymás utáni sorrendjére alapozott műveletek, mint például a gördülő (kumulált) összegek számítása pénzügyi jelentéseken; különösen régebbi szervereken, melyek nem támogatják az ilyen irányú SQL DML kiegészítéseket. Bonyolult mintázatok, például időbeli mintázatok keresése esetén azonban ez a támogatás is kevés lehet.
- Ha a rekordokat egyes mező-értékektől függő módon egészen más műveletekkel, de sorrendben kell feldolgozni, például egyik rekord egy rendelést, másik egy email küldést eredményez stb.
- Speciális alkalmazások, például dinamikus SQL-t tartalmazó scriptek generálása

NE használjunk kurzort olyan esetben, amikor a feladat áttekinthető módon megoldható a standard halmaz-alapú SQL DML utasításokkal, vagy ha a rekordhalmaz várhatóan nagy méretű (több millió rekordos)[7].

A kurzorok típusai

A forrás-rekordhalmaz leválasztottsága szempontjából (nagy betűkkel zárójelben a T-SQL kulcsszavai):

- Statikus (STATIC): a kurzor megnyitásakor a rekordhalmaz a tempdb adatbázis táblájaként jön létre, ezzel leválik a forrásról, és a forrás esetleges futás közbeni változásai már nem látszanak benne.

[6] https://msdn.microsoft.com/en-us/library/ms181441(v=sql.110)
[7] https://stackoverflow.com/questions/3861558/what-are-the-benefits-of-using-database-cursor

- Keyset (KEYSET): a létrehozás után beszúrt rekordok nem látszanak, de a meglévő rekordok változásai igen
- Dinamikus (DYNAMIC): minden változás látszik

A mutató mozgathatósága szempontjából:

- Csak előre mozgatható (FORWARD_ONLY): csak a FETCH NEXT alkalmazható
- Szabadon mozgatható (SCROLL): ekkor a FETCH FIRST, LAST, PRIOR, NEXT, RELATIVE, ABSOLUTE bármelyike alkalmazható

A kurzor láthatósága szempontjából:

- Lokális (LOCAL): a kurzor csak abban a kötegben, tárolt eljárásban vagy triggerben elérhető (hivatkozható), melyben létrehozták
- Globális (GLOBAL): a kurzor az aktív kapcsolat bármelyik kötegéből, tárolt eljárásából vagy triggeréből elérhető

Speciális, teljesítményre optimalizált, „foreach" típusú feladatokra gyakran használt kurzortípus a FAST_FORWARD: ez csak előre mozgatható, és a mutató által mutatott rekord a kurzoron keresztül nem módosítható (ez utóbbi részleteit nem tárgyaljuk). Természetesen a dinamikus és mozgatható kurzorok igénylik a legtöbb szerver oldali erőforrást.

PÉLDÁK kurzor-típusokra

```
--példa dinamikus, mozgatható kurzorra
go
declare c cursor global dynamic scroll for select LastName from Employees
print @@cursor_rows --eredm: 0, nincs nyitott kurzor
open c
print @@cursor_rows --eredm: -1, mivel dinamikus, nem megállapítható a sorok száma
--de ha volna egy order by LastName a forráson, a rendezéshez létrehozná a táblát,
--és a cursor_rows is 10 értéket adna vissza
declare @name varchar(50)
fetch last from c into @name  --fetch absolute nem lehet dinamikus kurzoron
print @name --Lamer
close c
deallocate c
--példa statikus kurzorra
go
declare c cursor global static scroll for select LastName from Employees
open c
print @@cursor_rows --10
declare @name varchar(50)
fetch absolute 2 from c into @name
print @name --Fuller (rendezés nélkül)
close c
deallocate c
```

Ha nem adjuk meg a kurzor típusát, akkor a T-SQL-ben általában globális, dinamikus és mozgatható kurzor jön létre, de a default kurzortípus magától a lekérdezéstől is függ, ezért *célszerű* mindig meghatározni a típust.

PÉLDA: a korábbi mező-orientált -> rekord-orientált átalakítás kurzorral

```
declare @emp_id int, @emp_name nvarchar(50), @i int, @address nvarchar(60),
    @city nvarchar(50)
create table #eredm (adatid int, dolg_id int,
```

```
        adatnev varchar(50), adatertek nvarchar(100))
declare cursor1 cursor fast_forward
    for
    select employeeid as emp_id, firstname+' '+lastname as emp_name,address, city
    from employees
set @i=0
open cursor1
fetch next from cursor1 into @emp_id, @emp_name, @address, @city
while @@fetch_status = 0
begin
    --print @emp_id print @emp_name print @i
    if @emp_name is not null begin
        insert #eredm values(@i, @emp_id, 'dolgozo_neve', @emp_name)
        set @i=@i+1
    end
    if @address is not null begin
        insert #eredm values(@i, @emp_id, 'dolgozo_cime', @address)
        set @i=@i+1
    end
    if @city is not null  begin
        insert #eredm values(@i, @emp_id, 'dolgozo_varosa', @city)
        set @i=@i+1
    end
    fetch next from cursor1 into @emp_id, @emp_name, @address, @city
    --print @emp_id
end
close cursor1
deallocate cursor1
print @i
print 'feldolgozás vége'
select * from #eredm
drop table #eredm
GO
```

ÖNÁLLÓ FELADAT #13: A feladat annak kiderítése, hogy a paraméterként kapott ügynök 1) napokban mérve milyen átlagos gyakorisággal köt nagy értékű rendelést, illetve 2) hányszor fordult elő vele, hogy két kis értékű rendelése közvetlenül követte egymást. Nagy értékűnek a 200 dollár feletti rendelések számítanak.

Például tegyük fel, hogy egy ügynök rendeléseinek a dátumai és összegei az alábbiak:

1. 2011. jan. 3. (820 $)
2. 2011. jan. 4. (190 $)
3. 2011. jan. 5. (11,200 $)
4. 2011. jan. 10. (100 $)
5. 2011. jan. 11. (140 $)
6. 2011. jan. 20. (540 $)

Ekkor egyrészt 17 nap alatt 3 nagy értékű rendelés volt, tehát az átlag 5.7 nap, másrészt egyszer fordult elő, hogy egymás után két kis értékű rendelése jött.

A megoldás javasolt lépései:

1. kurzor készítése az illető ügynök rendelési dátumaira és azok értékére, az előző feladat alapján, rendezés dátum szerint
2. a kurzor iterálása, a rendelés típusának mentése két rekord között egy változóba
3. tesztelés az Orders táblán

Segítség: SELECT DATEDIFF(day, '2011-02-23', '2011-03-03') -- az eredmény 11, integer

ÖNÁLLÓ FELADAT #14 (szorgalmi): hívjuk meg sorban minden ügynökre két egymásba ágyazott kurzorral úgy, hogy a külső az ügynökökön iterál végig, a belső az illető rendelésein.

ÖNÁLLÓ FELADAT #15 (szorgalmi): Elemezzük a rendelési tételek idősorát, hogy valós képünk legyen a szezonálisan változó keresletről. Állapítsuk meg, voltak-e olyan *legalább* kéthetes, *legalább* 5 rendelést tartalmazó időszakok, mikor *legalább* a rendelések 70%-án szerepelt például a Beverages termékkategória valamelyik terméke! Ha találunk ilyen periódusokat, írjuk ki az időszakok kezdő és záró dátumát, és a napok és a rendelések számát!

Például: „Időszak eleje: 1996-07-10, vége: 1996-07-29, napok száma: 19, rendelések száma: 15, arány: 73%”

A script paraméterei: az érdekes kategória azonosítója (1), az időszak minimális hossza (14), a rendelések minimális száma az időszakon belül (5), a minimális arány (70%).

Analitikus függvények és ablakozás

A legtöbb modern SQL-változat által támogatott úgynevezett analitikus vagy ablakozó (windowing) függvények az esetek egy részében kiválthatják a kurzor használatát, mivel a rekordok egy csoportja, partíciója alapján számolt értékeket csoportosítás (GROUP BY) nélkül is lekérdezhetővé teszik. Ezáltal a csoportosított és nem csoportosított lekérdezések lehetőségei egyszerre használhatók ki. Egy lekérdezésen belül mezőnként többféle szempontú particionálás is megvalósítható. Az analitikus függvényeket a lényeges technológiák kisebb eltérésekkel támogatják[8]. An analitikus függvényeket általában a SELECT listában használjuk, a következő szintaxissal: FÜGGVÉNY_NEVE([paraméter]) OVER ([PARTITION BY particionálási_kifejezés] [ORDER BY rendezési_kifejezés] [ROWS ablak_specifikáció]). Az ablak-specifikációban szerepelhetnek relatív pozícionálási kulcsszavak, pl. CURRENT ROW, PRECEDING, FOLLOWING, bővebben lásd a példában.

A lényegesebb függvények:

- FIRST_VALUE, LAST_VALUE(kifejezés) OVER…: a rendezett lista/partíció első/utolsó rekordjából számított kifejezés
- ROW_NUMBER() OVER…: a rendezett lista/partíció minden rekordjához egy futó sorszámot ad
- LAG(kifejezés, [offset], [default]) OVER…: a rendezett lista/partíció aktuális rekordja előtti offset számú rekord alapján számítja a kifejezést. Jól használható idősorok elemzéséhez.
- PERCENT_RANK() OVER…: a rendezett lista/partíció minden rekordjához megadja a rendezési feltétel alapján számított rangot, tehát a relatív pozíciót 0 és 1 közötti szám formájában

A fentieken kívül az SQL Server 2012 óta a szokásos aggregáló függvények (MIN, MAX, AVG, SUM stb.) is használhatók az ablakot definiáló OVER utótaggal.

PÉLDA: a #13. feladat megoldása ablakozással

```
go
drop table #tmp --az átláthatóság érdekében használjunk egy ideiglenes táblát
go
select orderdate, cast(sum((1-discount)*unitprice*quantity) as money) as value
into #tmp from orders o inner join [order details] d on o.orderid=d.orderid
```

[8] https://docs.microsoft.com/en-us/sql/t-sql/functions/analytic-functions-transact-sql?view=sql-server-2017
https://docs.microsoft.com/en-us/sql/t-sql/queries/select-over-clause-transact-sql?view=sql-server-2017
https://oracle-base.com/articles/misc/analytic-functions

```
where EmployeeID = 2
group by o.orderid, o.orderdate --96
order by OrderDate
go
select * from #tmp order by OrderDate

select orderdate, max(value) over
       (order by orderdate rows between 1 preceding and current row) as maxi-- maxi: a
rendelés és az előző rendelés közül a nagyobbik érték
from #tmp

--ahol ebben a listában 200 alatti érték van, ott 2 kicsi jött egymás után:
select a.OrderDate, a.maxi
from (
       select orderdate, max(value) over
              (order by orderdate rows between 1 preceding and current row) as maxi
       from #tmp
) as a where a.maxi < 200  --3 db
--MEGJEGYZÉS:
--vigyáznunk kell, mert a lista elején tévesen emelnénk ki a rekordot, ha már az első is
kis értékű
--helyesebben:

select a.OrderDate, a.maxi
from (
       select orderdate,
       max(value) over
              (order by orderdate rows between 1 preceding and current row) as maxi,
       row_number() over
              (order by orderdate) as sorsz
       from #tmp
) as a where a.maxi < 200  and sorsz > 1
```

ÖNÁLLÓ FELADAT #16: módosítsuk a fenti példa lekérdezését úgy, hogy az időben egymás után következő **5 nagy értékű** (1000 feletti) rendelést keressük!

A **partíciónként** számított kumulált értékek különösen pénzügyi jelentésekben lehetnek hasznosak.

PÉLDA: partíciónként vagy mozgó rekord-környezetben számított értékek, gördülő összeg, sorszám stb. évente:

```
select year(orderdate) ev,
       row_number() over (
              partition by year(orderdate)
              order by orderdate) sorszam,  --éven belüli futó sorszám
       cast(orderdate as date) datum,
       value osszeg,
       sum(value) over (
              partition by year(orderdate)
              order by orderdate
              rows between unbounded preceding and current row) eves_kumulalt_osszeg, --
éven belüli kumulált összeg
       avg(value) over (
              order by orderdate
              rows between 3 preceding and 3 following) mozgo_atlag_osszeg,
       sum(value) over (
              partition by year(orderdate) ) eves_total --az év teljes összege
from #tmp
```

ÖNÁLLÓ FELADAT #17: módosítsuk a #16. feladat lekérdezését úgy, hogy az időben egymás után következő 5 nagy értékű (1000 feletti) rendelést keressük, de **ügynökönként külön partícióban**!

```
--melyik évben melyik volt a legkisebb/legnagyobb értékű rendelés (al-lekérdezés nélkül,
dátummal és értékkel)
drop table #tmp2
go
select o.*, v.value
into #tmp2  --rendelések értékei a value mezőben
from orders o inner join
       (select orderid, cast(sum((1-discount)*unitprice*quantity) as money) as value
       from [order details] group by orderid ) as v
on v.orderid = o.orderid

select distinct year(orderdate), --a distinct miatt évente 1 rekord lesz
       first_value(orderdate) over (partition by year(orderdate) order by value asc) as
min_datum,
       first_value(orderid) over (partition by year(orderdate) order by value asc) as
min_rend_id,
       first_value(value) over (partition by year(orderdate) order by value asc) as
min_ertek,
       first_value(orderdate) over (partition by year(orderdate) order by value desc) as
max_datum,
       first_value(orderid) over (partition by year(orderdate) order by value desc) as
max_rend_id,
       first_value(value) over (partition by year(orderdate) order by value desc) as
max_ertek
from #tmp2
```

4. Tranzakció-kezelés T-SQL-ben

A **tranzakció** az üzleti folyamatnak a felhasználó szempontjából értelmes funkcionális egysége. Ez a funkcionális egység további funkcionális egységeket tartalmazhat, más szóval a tranzakciók egymásba ágyazhatók. Ha például tranzakciónak egy számla banki kiegyenlítését tekintjük, ezen belül tekinthetjük beágyazott tranzakciónak az egyik fél számlájának terhelését az egyik bank információs rendszerében, és egy másik beágyazott tranzakciónak az összeg jóváírását a másik bank információs rendszerében, hiszen mindkét beágyazott tranzakció maga is sok lépésből áll.

A tranzakciókat futtató információs rendszerektől elvárható, hogy biztosítsák a tranzakciók négy alap-tulajdonságát (ún. ACID-tulajdonságok):

- **Atomicitás**: elvárjuk, hogy a tranzakció minden lépése sikeresen lefusson, vagy pedig hiba esetén a már végrehajtott lépések visszavonásra kerüljenek. Ez az elvárásunk nem teljesült, mikor a telefonos rendelési példában programozási hiba miatt nem tudtuk felvenni a rendelést, de a vevő számláját azért megterheltük: a tranzakció atomicitása sérült.
- **Konzisztencia**: elvárjuk, hogy az adatbázis folyamatosan konzisztens állapotban maradjon. A programozási vagy egyéb hibák nem vezethetnek például külső kulcs kényszerek sérüléséhez.
- **Elszigetelés (izoláció)**: elvárjuk, hogy az egymással párhuzamosan futó tranzakciók a lehető legkevésbé legyenek hatással egymásra.
- **Tartósság**: elvárjuk, hogy a sikeresen lefutott tranzakciók hatása hardver hiba esetén is megőrződjön.

Az adatbázis-kezelő rendszerek az elszigetelést a zárolással, a többi három tulajdonságot a tranzakció-naplóval (transaction log) biztosítják.

Az atomicitás biztosítása

Minden adatmódosító SQL utasítás a programozó által nem deklarált, ún. **implicit tranzakciónak** tekinthető.

PÉLDA implicit tranzakcióra

```
create table t1 (id int primary key)
create table t2 (id int primary key, t1_id int references t1(id))
go
insert t1 (id) values (1), (3), (4), (5)
insert t2 (id, t1_id) values (10, 3) --a (3) rkord nem törölhető
go
delete t1 --implicit tr.
--"The DELETE statement conflicted with the REFERENCE constraint ..." etc
select * from t1 –minden rekord megvan
```

A fenti példában a szerver visszagörgette az első t1 rekord törlését a második rekord törlésekor bekövetkezett hiba miatt. Annak érdekében, hogy az adatbázis tartalmát módosító műveletek visszagörgethetőek legyenek, a szerver speciális, és sok erőforrást igénylő **tranzakciós naplót** és speciális naplózási mechanizmust (Write Ahead Log, WAL) használ. A naplóban minden tranzakció minden adatmódosító utasításának van egy-egy, logikai időrendbe rendezett és sorszámozott bejegyzése, mely lehetővé teszi a lépés visszagörgetését ('roll back') vagy újbóli lefuttatását ('roll forward'). Ez az alapja a több utasításból álló, programozott tranzakciók visszagörgetésének is. Az újbóli lefuttatásra adatvesztés esetén lehet szükség. A megvalósítás részleteit ez a jegyzet nem tárgyalja.

Az explicit (programozott) tranzakciók kezelését az SQL szabvány a BEGIN/COMMIT/ROLLBACK TRANSACTION utasításokkal támogatja. A tranzakció a BEGIN utasítással kezdődik, és a COMMIT vagy ROLLBACK utasítással ér véget. A COMMIT felszabadítja a tranzakció által lefoglalt erőforrásokat, mint például a zárak különféle objektumokon. A ROLLBACK ezen kívül a tranzakciós napló alapján visszagörgeti az összes olyan változást is, melyet a tranzakció okozott az adatbázisban.

A T-SQL-ben az XACT_ABORT kapcsolati változó határozza meg, mi történjen, ha a tranzakció valamelyik utasítása hibára fut. A default beállítás, OFF esetén folytatódik a végrehajtás. ON esetén a rendszer visszagörgeti a tranzakciót.

PÉLDA: a rendszer által automatikusan visszagörgetett explicit tranzakcióra

```
set xact_abort off
delete t2
go
begin tran
       insert t2 (id, t1_id) values (10, 1)
       insert t2 (id, t1_id) values (11, 2) –külső kulcs hiba
       insert t2 (id, t1_id) values (12, 3)
commit tran
go
--"The INSERT statement conflicted with the FOREIGN KEY constraint ..." etc
select * from t2
id     t1_id
10     1
12     3
--az atomicitás sérült
set xact_abort on
delete t2
go
begin tran
       insert t2 (id, t1_id) values (10, 1)
       insert t2 (id, t1_id) values (11, 2) –külső kulcs hiba
       insert t2 (id, t1_id) values (12, 3)
commit tran
go
--"The INSERT statement conflicted with the FOREIGN KEY constraint ..." etc
select * from t2
id     t1_id
-- az atomicitás nem sérült
```

Egy valódi tranzakcióban nem csak programozási és integritási, hanem **üzleti logikai hibák** esetén is használjuk a tranzakció programozott visszagörgetésének a lehetőségét. Ilyen logikai hiba lehet például a hiányzó fedezet.

PÉLDA: a korábbi rendelés-felvétel megvalósítása egy explicit tranzakcióban, programozott visszagörgetéssel

```
--rendelés-felvétel tranzakció-kezeléssel
declare @prod_name varchar(20), @quantity int, @cust_id nchar(5) --a telefonból (a vevő-ID
szöveges)
declare @status_message nvarchar(100),  @status int --a folyamat visszatérési értéke
declare @res_no int --találatok száma
declare @prod_id int, @order_id int --azonosítók
declare @stock int --raktárkészlet
declare @cust_balance money --a vevő egyenlege
declare @unitprice money --a termék egységára

-- paraméterek
```

```
set @prod_name = 'boston'
set @quantity = 10
set @cust_id = 'AROUT'
begin tran
begin try
	select @res_no = count(*) from products where productname like '%' + @prod_name + '%'
	if @res_no <> 1 begin
		set @status = 1
		set @status_message = 'HIBA: a terméknév nem egyértelmű.';
	end else begin
		-- ha egy terméket találunk, megkeressük a kulcsot és a raktárkészletet
		select @prod_id = productID, @stock = unitsInStock from products where
productName like '%' + @prod_name + '%'
		-- van-e raktáron? a rendelt mennyiség elérheto-e?
		if @stock < @quantity begin
			set @status = 2
			set @status_message = 'HIBA: a raktárkészlet nem fedezi a megrendelést.'
		end else begin
		-- mennyibe kerül? Van elég pénze a vevönek?
			select @cust_balance = balance from customers where customerid =
@cust_id
								--ha nem találjuk a vásárlót, akkor az egyenleg
null
								--mivel kulcsra keresünk, max. 1 találat lehet
			select @unitprice = unitPrice from products where productID = @prod_id -
-nincs discount
			if @cust_balance < @quantity*@unitprice or @cust_balance is null begin
				set @status = 3
				set @status_message = 'HIBA: a vásárló nem található, vagy az
egyenlege túl alacsony.'
			end else begin
		-- Ellenőrzések vége, elkezdjük a tranzakciót (2 lépés)
		-- 1. vásárló egyenlegének frissítése
				update customers set balance = balance-(@quantity*@unitprice)
where customerid=@cust_id
		-- 2. új bejegyzés az Orders, Order Details táblákba
				insert into orders (customerID, orderdate) values (@cust_id,
getdate()) --orderid: identity
				set @order_id = @@identity  --az utolsó identity insert eredménye
				insert [order details] (orderid, productid, quantity, UnitPrice)
					--itt a hiba
				values(@order_id, @prod_id, @quantity, @unitprice)
					--itt a hiba
--				insert [order details] (orderid, productid, quantity, UnitPrice,
Discount)
					--ez a jó
--                                      values(@order_id, @prod_id, @quantity, @unitprice, 0)
                                        --ez a jó
                                set @status = 0
                                set @status_message = cast(@order_id as varchar(20)) + ' sz.
rendelés sikeresen felvéve.'
                        end
                end
        end
        print 'Állapot: ' + cast(@status as varchar(50))
        print @status_message
        if @status = 0 commit tran else begin
                print 'Visszagörgetve'
                rollback tran
        end
end try
begin catch
```

```
        print 'EGYÉB HIBA: '+ ERROR_MESSAGE() + ' (' + cast(ERROR_NUMBER() as varchar(20)) +
')'
        rollback tran
end catch
go


--teszteléshez beállítjuk a paramétereket
set nocount off
update customers set balance=1000 where CustomerID='AROUT'
delete [Order Details] where OrderID in (select orderid from Orders where CustomerID='AROUT'
and EmployeeID is null)
delete Orders where CustomerID='AROUT' and EmployeeID is null
--most futtatjuk a scriptet, utána ellenőrzés:
select * from Customers where CustomerID='AROUT'
select top 3 * from Orders where CustomerID='arout' order by OrderDate desc
--a programozási hiba előtti update és insert utasítás visszagördült
```

ÖNÁLLÓ FELADAT #18: valósítsuk meg az alábbi egyszerű tranzakciót, melynek paraméterei a termék neve és a termék kategóriájának ID-je (a Products tábla ProductName és CategoryID mezői). A lépések:

- új termék beszúrása a kapott névvel a Products táblába
- annak ellenőrzése, hogy a kapott kategóriában nincs-e már 10-nél több termék
- ha van, visszagörgetjük a tranzakciót (ROLLBACK)
- ha nincs, az újonnan felvett termék raktárkészletét beállítjuk 100-ra
- A tranzakció befejezése (COMMIT)

Ellenőrizzük a visszagörgetés megtörténtét!

SZORGALMI FELADAT: adjunk tranzakcionális támogatást a korábbi raktárkészlet-aktualizálós scriptünknek és teszteljük logikai és egyéb hibákra.

A T-SQL tranzakciók **egymásba ágyazhatók**, mint az a valóságban el is várható. A programozónak figyelnie kell rá, hogy minden tranzakció kezdete után COMMIT is következzen, vagy pedig egy ROLLBACK, mely minden megnyitott tranzakciót visszagörget. Ha egy tranzakció nyitva marad, és a kliens kapcsolata az adatbázis-szerverrel nem szakad meg, akkor a tranzakció folyamatosan foglalja a szerver erőforrásait, amíg a kapcsolat fennáll, és ez idővel **súlyos hibákat** okozhat. Ennek elkerülését a T-SQL a @@TRANCOUNT globális változóval támogatja.

PÉLDA: egymásba ágyazott tranzakciókra

```
begin tran
        print @@trancount  --1
        begin tran
                print @@trancount  --2
        commit tran
        print @@trancount  --1
commit tran
print @@trancount  --0
go
begin tran
        print @@trancount  --1
        begin tran
                print @@trancount  --2
rollback tran
print @@trancount  --0
```

Az izoláció biztosítása

Az izoláció biztosítására a szerver a közös erőforrásként használt objektumokra (rekordokra, tartományokra vagy egész táblákra, adatbázisokra) elhelyezett **zárakat** használ. A zár olyan adatszerkezet, melynek a tulajdonosa egy tranzakció, egy objektumra vonatkozik, van típusa és van állapota: elnyert (grant) vagy várakozó (wait). Az SQL server több mint 20 különböző zár-típust használ, melyek közül a legfontosabbak:

- **Megosztott** (shared, S): a SELECT-hez használt, olvasási zár
- **Kizárólagos** (exclusive, X): az adatmódosító SQL utasításokhoz használt zár. A tranzakció által megszerzett X típusú zárak *mindig megőrződnek a tranzakció befejezéséig*. Erre a tranzakció logikai egységének biztosítása céljából van szükség (az ún. *lost update* elkerülése céljából).
- **Update** (U): ideiglenes zár, melyet az adatmódosításra készülő tranzakció a szűrőfeltétel kiértékelésének idejére szerez meg. Ha a feltétel egy rekordra teljesül, akkor X típusú zárrá próbálja alakítani.

A zár attól függően lesz elnyert vagy várakozó állapotú, hogy milyen, más tranzakció által elhelyezett zárak vannak már az illető objektumon. **Elnyert** lesz az állapot, ha a tranzakció

- S zárat S zár vagy U zár mellé helyez,
- X zárat teljesen szabad (semmilyen, más tranzakcióhoz tartozó zárral sem védett) objektumra helyez,
- U zárat S zár mellé helyez el.

Egyéb esetben a zár elhelyezése után a tranzakció erőforrásra váró (felfüggesztett vagy blokkolt) állapotba kerül, egészen addig, amíg a blokkolást okozó zárak meg nem szűnnek.

A zárak elhelyezését és eltávolítását SQL utasítás szinten is vezérelheti a programozó a tábla neve után elhelyezett direktívával[9], de kényelmesebb és biztonságosabb ezt a rendszerre bízni. Az SQL szabvány 4 zárelhelyezési/megszüntetési stratégiát definiál az S zárakra, melyeket **izolációs szinteknek** szokás nevezni:

- **Read uncommitted** (más néven „dirty read”): a SELECT utasítások nem helyeznek el S zárat, ezért egy másik tranzakcióhoz tartozó X zár nem blokkolja az olvasást.
- **Read committed**: a tranzakció az olvasás idejére S zárat szerez (ha tud), majd a sikeres olvasás után azonnal megszünteti. Ezért az olvasás után nem blokkolja más tranzakciók módosító utasításait (melyhez X zár elnyerése kell). Bármilyen technológiájú kliens csatlakozik is az adatbázishoz, ez a **default izolációs szint**.
- **Repeatable read**: mint az előző, azonban az S zár a tranzakció végéig megmarad, és blokkolja más tranzakciók módosító utasításait.
- **Serializable**: az S zárak nemcsak az olvasott rekordra, hanem az egész tartományra a tranzakció végéig megmaradnak, ezért más tranzakciók nem tudnak a tartományba rekordokat beszúrni/törölni sem. Tehát a „semmiből” megjelenő vagy eltűnő rekord (az ún. *phantom read* jelenség) nem fordulhat elő.

[9] Ezek a 4 izolációs szint sorrendjében a NOLOCK, READCOMMITTED, REPEATABLEREAD és a HOLDLOCK. Akár egyetlen SQL utasításon belül is minden táblára külön direktíva (table hint) vonatkozhat.

Az izolációs szint a kapcsolat (session) paramétere, melyet a SET TRANSACTION ISOLATION LEVEL … utasítással lehet átállítani.

A fenti 4 ISO szabványos szinten kívül az egyes gyártók további szinteket is támogatnak, speciálisan az SQL Server 2012 utáni kiadásai kettőt: a Snapshot és a Read committed snapshot szintet. Ezek részleteit nem tárgyaljuk.

PÉLDA tranzakciók **blokkolására**. Egy editorban indítsunk el egy adatmódosító tranzakciót:

```
set transaction isolation level read committed  --vagy bármelyik másik
begin tran
update Employees set Salary = 1000 where EmployeeID = 1

select @@SPID --63 (session ID)
select resource_type, request_mode, request_type,
       resource_associated_entity_id, request_status, request_session_id
from sys.dm_tran_locks where request_session_id=63 --63 helyett a saját session ID-nk
```

Mint az a session-hoz tartozó zárak listáján látszik, ez a tranzakció egy X zárat helyez el a tábla első rekordjára, és amíg a tranzakció véget nem ér, a zár meg is marad *mind a négy izolációs szinten.* A tranzakció lassú futását most úgy szimuláljuk, hogy egyelőre nincs ROLLBACK vagy COMMIT. Ezért egy másik editorban kiadott SELECT a lekérdezés típusától függően várakozni kényszerül:

```
select * from Employees where EmployeeID = 2  --lefut, mert az ID-re van index
select * from Employees where FirstName = 'Andrew'  --várakozik, mert minden rekordot meg kell
néznie
```

Ezután ne feledjük lezárni a tranzakciót egy ROLLBACK utasítással.

Az, hogy az egyes tranzakciók egy konkrét alkalmazás és egy konkrét izolációs szint esetén mennyi időt töltenek blokkolt (várakozó) állapotban, különféle diagnosztikai eszközökkel ellenőrizhető. Az adatbázis-alkalmazások gyorsításának egy lehetséges módja ügyes tervezéssel ezen várakozási idők minimalizálása.

PÉLDA (*kiegészítő jellegű, CSAK ha van rá idő, kedv stb.*) arra, hogyan oldja meg az update zár az adatmódosító tranzakciók „kiéheztetésének” az elkerülését a folyamatosan olvasott táblákon. Három tranzakciót futtatunk három editor ablakban:

```
--ebben az editor ablakban:
set transaction isolation level serializable --tehát az S zár marad
begin tran
select * from Employees where FirstName = 'Andrew'  --range S zár marad
select distinct resource_type, request_mode, request_type,
       resource_associated_entity_id, request_status, request_session_id
from sys.dm_tran_locks where request_session_id=@@SPID --S lock a táblára: GRANT

--másik editorban kérdezzük le a @@SPID-t, legyen most ez 51
--utána a másik editorban futtassuk ezt:
set transaction isolation level serializable --S zár marad
begin tran
update Employees set Salary = 1000 where EmployeeID = 1
--várakozik

--ebben az editorban ellenőrizzük a másik, várakozó tr. zárjait:
select distinct resource_type, request_mode, request_type,
       resource_associated_entity_id, request_status, request_session_id
from sys.dm_tran_locks where request_session_id=51
--megjelenik a KEY X típusú WAIT állapotú zár

--ezek után egy harmadik editor ablakban:
```

```
set transaction isolation level serializable --S zár marad
begin tran
select * from Employees where EmployeeID = 1
--szintén várakozik, mert az X zár mellett az S zárat nem tudja megkapni
--így kerülhető el az adatmódosító tr. kiéheztetése: Ha az 1. tr. lefut, utána
--a 2. megkapja majd az X zárat, aztán jöhet a 3. tr. olvasása:

--ebben az ablakban:
commit tran
--erre a 2. ablakban lefut az update, de a 3. tr. még mindig várakozik

--ekkor a 2. ablakban:
commit tran

--erre az 1. ablakban is lefut a select
--tehát a később jött select-nek meg kellett várnia az update végét
--ne feledjük a 3. ablakban is:
commit tran
```

PÉLDA: **deadlock** (holtpont) demonstrálása egy webáruház esetén. A script először ellenőrzi, hogy a vevő által kiválasztott cikk (jelen esetben egy ló) állapota szabad-e még. Erre azért van szükség, mert lehet, hogy közben egy konkurens tranzakcióban valaki más már megvette. Ha szabad, akkor terheli a vevő bankszámláját, ami 10 mp késleltetést jelent, majd átállítja a cikk állapotát eladottra (sold), és beírja a vevő nevét a rekordba.

```
create table test_product(id int primary key, prod_name varchar(50) not null, sold
varchar(50), buyer varchar(50))
insert test_product(id, prod_name, sold) values (1, 'car', 'for sale')
insert test_product(id, prod_name, sold) values (2, 'horse', 'for sale')
go
select * from test_product
update test_product set sold='for sale', buyer=null where id=2
go
set tran isolation level read committed -- default
go
begin tran
declare @sold varchar(50)
select @sold=sold from test_product where id=2
if @sold='for sale' begin
    waitfor delay '00:00:10' –terheljük a bankszámlát
    update test_product set sold='sold', buyer='My name' where id=2
    print 'sold successfully'
end else print 'product not available'
commit tran
go
--a fenti tranzakciót párhuzamosan két editorban futtatjuk
--a másik script:
set tran isolation level read committed
go
begin tran
declare @sold varchar(50)
select @sold=sold from test_product where id=2
if @sold='for sale' begin
    waitfor delay '00:00:10'
    update test_product set sold='sold', buyer='Your name' where id=2 –ez a 2. vásárló
    print 'sold successfully'
end else print 'product not available'
commit tran
go
--ez történik:
```

```
select * from test_product
id      prod_name      sold          buyer
1       car                    for sale      NULL
2       horse          sold          Your name
```

A példában mindkét eladás sikeresen megtörténik, tehát mindkét bankszámlát terheljük, de a lovat természetesen csak az egyik vásárló (mégpedig a második) fogja megkapni. A fenti példa tehát komoly logikai hibát eredményez az alacsony izolációs szint (read committed) miatt.

A fenti példában az események sorrendje a következő volt:

1. A 'My name' tranzakció a SELECT idejére S zárat nyert, melyet a SELECT után megszüntetett, majd 10 másodpercig várakozni kezdett.
2. A 'Your name' tranzakció a SELECT idejére S zárat nyert, melyet a SELECT után megszüntetett, majd 10 másodpercig várakozni kezdett.
3. A 'My name' tranzakció befejezte a várakozást, az UPDATE céljára X zárat nyert (mivel a rekordon semmilyen zár nem volt), beírta a táblába vevőnek a My name-t, majd a COMMIT után befejeződött. Ezzel megszűnt az általa elhelyezett X zár is.
4. A 'Your name' tranzakció is befejezte a várakozást, az UPDATE céljára X zárat nyert (mivel a rekordon már semmilyen zár nem volt), beírta a táblába vevőnek a Your name-t, majd a COMMIT után befejeződött. Ezzel megszűnt az általa elhelyezett X zár is.

Ugyanezt a scriptet a repeatable read izolációs szinten futtatva azt tapasztaljuk, hogy az egyik tranzakció sikeresen lefut (sold successfully), a másik viszont néhány másodperc plusz várakozás után hibára fut az alábbi hibaüzenettel:

*'Transaction (Process ID …) was **deadlocked** on lock resources with another process and has been chosen as the deadlock victim. Rerun the transaction.'*

Mivel a hibára futott tranzakciót a rendszer visszagörgeti, ezért csak annak a vásárlónak a számláját terheltük, akinek a nevét a táblába írtuk, és akinek pozitív visszajelzést adtunk. Tehát ezen az izolációs szinten **nem** követtünk el logikai hibát[10].

Most az események sorrendje a következő volt:

1. A 'My name' tranzakció a SELECT idejére S zárat nyert, melyet a SELECT után sem szüntetett meg, majd 10 másodpercig várakozni kezdett.
2. A 'Your name' tranzakció a SELECT idejére S zárat nyert, melyet a SELECT után sem szüntetett meg, majd 10 másodpercig várakozni kezdett. Tehát most már két S zár van a rekordon.
3. A 'My name' tranzakció befejezte a várakozást, az UPDATE céljára X zárat helyezett el, de mivel a rekordon már volt két S zár, ezért a zár és a tranzakció várakozó állapotba került. Ebből egész addig nem tud továbbjutni, míg a Your name meg nem szünteti az S zárat.
4. A 'Your name' tranzakció is befejezte a várakozást, az UPDATE céljára X zárat helyezett el, de mivel a rekordon már volt két elnyert S zár és egy várakozó X zár, ezért az új zár és a tranzakció is

[10] Megjegyzés: ugyanezt az automatikus visszagörgetést értük volna el deadlock nélkül, ha az SQL Server speciális snapshot izolációs szintjét alkalmazzuk, amely a rekordok verziózására épül: „A snapshot transaction always uses optimistic concurrency control, withholding any locks that would prevent other transactions from updating rows. If a snapshot transaction attempts to commit an update to a row that was changed after the transaction began, the transaction is rolled back, and an error is raised (https://docs.microsoft.com/en-us/dotnet/framework/data/adonet/sql/snapshot-isolation-in-sql-server)

várakozó állapotba került. Ebből egész addig nem tud továbbjutni, míg a My name meg nem szünteti az S zárat.

5. A rendszer érzékelte, hogy a tranzakciók közti várakozási gráfban kör alakult ki, és néhány másodperc után megszüntette és visszagörgette az egyik tranzakciót. Ezzel megszűntek az illető tranzakció zárjai is.
6. A másik tranzakció X zárja elnyert állapotúvá változott, az UPDATE sikeresen lefutott és a tranzakció a COMMIT-tal véget ért.

A holtpont-feloldás nem csak két tranzakcióra működik.

DEMO: Most *minden hallgató* nyisson egy editort a test nevű adatbázisra, és másolja be oda az alábbi kódot, a '...' helyére a saját nevét írva. Egyszerre futtassa mindenki a scriptet. Pontosan egy hallgató lehet sikeres, a többi tranzakciót a rendszer visszagörgeti.

```
use test
set tran isolation level repeatable read
go
begin tran
declare @sold varchar(50)
select @sold=sold from test_product where id=2
if @sold='for sale' begin
    waitfor delay '00:00:10' --terheljük a bankszámlát
    update test_product set sold='sold', buyer='...' where id=2
    print 'sold successfully'
end else print 'product not available'
commit tran
go
```

A magasabb izolációs szinten általában

- ✗ gyengül a rendszer teljesítménye (amit a másodpercenként feldolgozott tranzakciók számával mérnek), a plusz adminisztráció miatt és mivel a hosszú élettartamú zárakra többet várnak a tranzakciók,
- ✓ viszont csökken a fentihez hasonló logikai hibák előfordulásának az esélye.

A nagy teljesítményt, mint előnyt a logikai hiba kockázatával, mint hátránnyal együtt kell mérlegelni. A legalacsonyabb szint (read uncommitted) használata is indokolt lehet, ha tudjuk, hogy a tranzakció futásának idején nem futnak olyan tranzakciók, melyek az olvasott táblákat módosítják. Erre példa lehet egy ügyviteli rendszer tábláinak adattárházba másolása hivatali időn kívül.

Az izolációs szint helyes megválasztása a programozó felelőssége és az alkalmazástól függ.

ONÁLLÓ FELADAT #19: idézzük elő a fenti módon a logikai hibát és a holtpontot két különböző izolációs szinten a rendelés-felvételi tranzakcióban, két párhuzamos editorban való futtatással, az ellenőrzések után betett időkésleltetéssel!

Tranzakciók és az OLTP adatbázisok szerkezete

A helyesen tervezett OLTP adatbázisok szerkezete általában megmutatja az adatbázis által kiszolgált legfontosabb tranzakciókat.

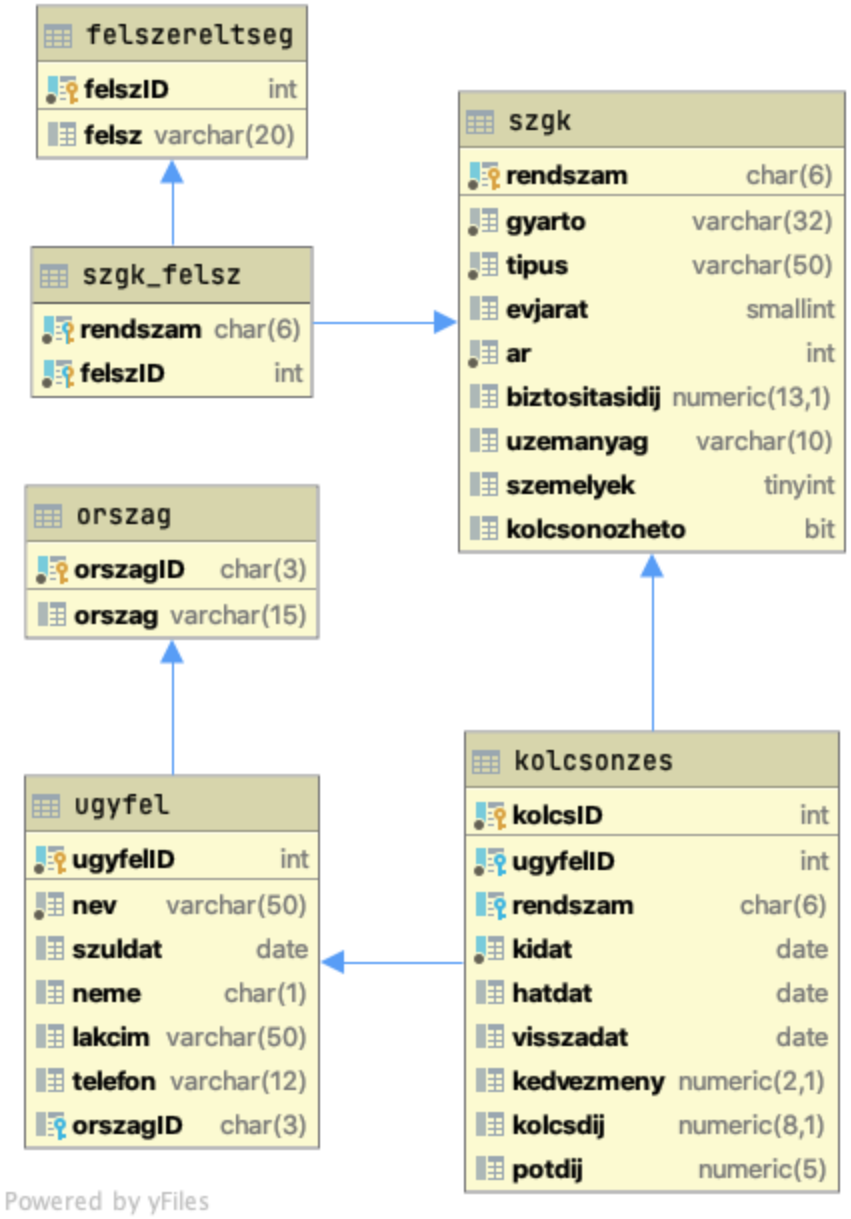
- Vannak olyan táblák, melyek nem hivatkoznak más táblára külső kulccsal. Ezeket **törzsállománynak** nevezzük.

- Ugyanakkor az adatbázisban legalább egy olyan tábla található, melyre nem hivatkozik más tábla külső kulccsal, viszont ezek sok (esetleg az összes) más táblára hivatkoznak közvetve vagy közvetlenül külső kulcsokkal. Az ilyen tábla az adatbázis (egyik) **tranzakcionális középpontja**.

A fenti strukturális jellemzőkön kívül a törzsállományok és a tranzakcionális középpont közti további jellegzetes különbségek:

- A törzsállományok általában kis méretűek és ritkán változnak. A középpont és a törzsállomány közti hivatkozási láncban minél közelebb van egy tábla a törzsállományhoz, annál inkább jellemző rá ez a tulajdonság.
- A középpont táblája nagy méretű és gyakran változik. Gyakran összetett kulcsa van, mert kapcsolótáblaként működik. Ez az a tábla, melyet a fő tranzakció folyamatosan módosít. A statikus jellegű törzsállományokkal szemben a középpont a legdinamikusabb, és a működés szempontjából a legkritikusabb rész.

PÉLDA: A Northwind adatbázisban a fenti strukturális definíció alapján törzsállományok a Categories, Suppliers, Shippers, Employees, Region táblák. A középpont az Order details tábla. Ez megfelel az elvárásainknak, mert az adatbázis által kiszolgált legfontosabb tranzakció a rendelés felvétele vagy módosítása. Szintén középpont jellegű az EmployeeTerritories tábla, melyhez tartozó esemény lehet egy alkalmazott adott területhez rendelése. Természetesen létezhetnek a törzsállományokat kezelő tranzakciók is a rendszerben, mint például a korábbi raktárkészlet-karbantartó scriptünk, azonban az üzleti folyamat szempontjából nem ezek a legfontosabbak és leggyakoribbak.



PÉLDA: a fenti db1 adatbázisban törzsállomány a felszereltseg, orszag, szgk tábla, középpont a kolcsonzes tábla. Nem meglepő, hogy a legfontosabb, leggyakoribb tranzakció egy autó kölcsönzése vagy egy kölcsönzés rekord módosítása.

5. Szerver oldali tárolt logika

Tárolt eljárások

A tárolt eljárások lehetőséget adnak az SQL scriptek adatokkal együtt történő menedzselésére és távoli eljárásként való meghívására. Az eljárást a CREATE PROCEDURE paranccsal hozhatjuk létre. Ennek hatására a kód nem fut le, hanem végrehajtási terv generálódik hozzá, és a szerveren tárolásra kerül, mint az adatbázis egy objektuma. Hasonló módon lehet a szerveren tárolt felhasználói függvényeket is létrehozni.

PÉLDA: emeljük meg a paraméterként kapott nevű dolgozó fizetését 10%-kal (most tranzakció-kezelés nélkül)

```
go
create procedure sp_inc_salary @name nvarchar(20) as
begin try
	set nocount on
	declare @address nvarchar(max), @res_no int, @emp_id int
	select @res_no=count(*) from Employees where LastName like @name
	if @res_no=0 print 'Nincs találat.'
	else if @res_no>1 print 'Több, mint 1 találat.'
	else begin  --épp egy találat
		select @address=Country+', '+City+' '+Address, @emp_id=EmployeeID
			from Employees where LastName like @name
		print 'Dolgozó ID: ' + cast(@emp_id as varchar(10)) + ', cím: ' + @address
		update Employees set salary=1.1*salary where EmployeeID=@emp_id
		print 'Egyenleg növelve.'
	end
end try
begin catch
	print 'EGYÉB HIBA: '+ ERROR_MESSAGE() + ' (' + cast(ERROR_NUMBER() as varchar(20)) +
')'
end catch
go
--teszt
execute sp_inc_salary 'Fuller'
```

Ellenőrizzük az új eljárás létrejöttét az adatbázis Programmability csoportjában!

ÖNÁLLÓ FELADAT #20: írjunk egy tárolt eljárást, amely a kapott azonosítójú termék termék-kategóriáját visszaírja

Az eljárás tartalmazhat minden eddig ismertetett T-SQL vezérlési elemet (például hibakezelés, tranzakció-kezelés, kurzorok stb.) és meghívhat más tárolt eljárásokat is az EXECUTE paranccsal.

A tárolt eljárások visszatérési értékei

Az eljárás több különböző módon adhat vissza értéket a kliens oldalnak:

- Üzenetek (PRINT), figyelmeztetések és hibák (RAISERROR) generálhatók
- Az eljárás paraméterei között lehetnek ún. OUTPUT paraméterek is, melyek „cím szerint” kerülnek átadásra, és a kliens a futás után az előzetesen hozzárendelt változóban fel tudja használni az eljárás által beállított értéket.

- Azok a SELECT utasítások, melyek eredményét nem tároljuk változókba, a kliensnek átadják a SELECT eredmény-rekordhalmazát. Egy eljárásnak több visszaadott rekordhalmaza is lehet. Ez a kényelem SQL Server specifikus, más rendszereken kurzor típusú OUTPUT paramétert kell használni ilyen célra.
- Minden eljárás visszaad egy int típusú értéket is, melyben tipikusan a sikeres vagy nem sikeres futást lehet kódolni. A programozó a RETURN utasítást használhatja az érték beállítására. Ennek hiányában a rendszer állítja be az értéket, hiba esetén az érték <0.

PÉLDA visszatérési értékre

```
drop proc p
go
create proc p as print 'sikeres futás'
go
declare @eredm int
exec @eredm=p
print @eredm --0

drop proc p
go
create proc p as
print 'sikeres futás'
return 1
go
declare @eredm int
exec @eredm=p
print @eredm --1

drop proc p
go
create proc p as select 1/0
go
declare @eredm int
exec @eredm=p
print @eredm --              -6 (<0, mert hiba történt)
```

PÉLDA arra, hogy az egyik eljárás hívja a másikat. A korábbi rendelés-felvételi tranzakciónk most egy tárolt eljárásban fut és egy másik eljárást hív meg, mely a rendelés tételei alapján aktualizálja a raktárkészletet. A raktárkészlet-hiba kezelését is erre a beágyazott tárolt eljárásra bízzuk.

```
drop procedure  sp_raktárkészlet_aktualizalo
go
create procedure sp_raktárkészlet_aktualizalo
@orderid int, --a rendelés azonosítója
@result varchar(50) output --visszajelzés
as
begin try
        update products set unitsInStock = unitsInStock - od.quantity
        from products p inner join [Order Details] od on od.ProductID=p.ProductID
        where od.OrderID=@orderid
        set @result='OK'
end try
begin catch --hiba akkor lehet, ha valamelyik tétel negatívba vinné a készletet
        print '  Készletezési hiba: '+ ERROR_MESSAGE() + ' (' + cast(ERROR_NUMBER() as
varchar(20)) + ')'
        set @result='HIBA'
end catch
go
```

```
--teszt
insert orders (OrderDate) values (getdate())
select @@identity --12105
select * from Products where ProductID=9 --unitsinstock =29
select * from Products where ProductID=10 --unitsinstock =31
insert [Order Details]  (orderid, productid, quantity, UnitPrice, Discount)
values (12105, 9, 10, 30, 0),(12105, 10, 40, 30, 0)
--tehát a második tétel hibát ad az sp_raktárkészlet_aktualizalo hívásakor:
go
declare @eredm varchar(50)
exec sp_raktárkészlet_aktualizalo 12105, @eredm output
print @eredm
go
--helyreállítás
delete [Order Details] where OrderID=12105
delete Orders where OrderID=12105
update Products set UnitsInStock=29 where ProductID=9
update Products set UnitsInStock=31 where ProductID=10
select * from Products

--a rendelés-felvételi tranzakció tárolt eljárásban
--a raktárkészlet ellenőrzését most a fenti eljárásra bízzuk
create procedure uj_rendeles
@prod_name varchar(20),
@quantity int,
@cust_id nchar(5) --a három bemeneti paraméter
as
declare @status_message nvarchar(100),  @status int --a folyamat visszatérési értéke
declare @res_no int --találatok száma
declare @prod_id int, @order_id int --azonosítók
declare @stock int --raktárkészlet
declare @cust_balance money --a vevő egyenlege
declare @unitprice money --a termék egységára

-- paraméterek
begin tran
begin try
        select @res_no = count(*) from products where productname like '%' + @prod_name + '%'
        if @res_no <> 1 begin
                set @status = 1
                set @status_message = 'HIBA: a terméknév nem egyértelmű.'
        end else begin
                -- ha egy terméket találunk, megkeressük a kulcsot
                select @prod_id = productID from products where productName like '%' +
@prod_name + '%'
--INNEN KIVETTÜK A RAKTÁR ELLENŐRZÉSÉT
                -- mennyibe kerül? Van elég pénze a vevönek?
                select @cust_balance = balance from customers where customerid = @cust_id
                                        --ha nem találjuk a vásárlót, akkor az egyenleg null
                                        --mivel kulcsra keresünk, max. 1 találat lehet
                select @unitprice = unitPrice from products where productID = @prod_id --nincs
discount
                if @cust_balance < @quantity*@unitprice or @cust_balance is null begin
                        set @status = 2
                        set @status_message = 'HIBA: a vásárló nem található vagy az egyenlege
túl alacsony.'
                end else begin
        -- Ellenőrzések vége, elkezdjük a tranzakciót (3 lépés)
        -- 1. vásárló egyenlegének frissítése
                update customers set balance = balance-(@quantity*@unitprice) where
customerid=@cust_id
```

```
        -- 2. új bejegyzés az Orders, Order Details táblákba
                        insert into orders (customerID, orderdate) values (@cust_id, getdate())
--orderid: identity
                        set @order_id = @@identity  --az utolsó identity insert eredménye
                        insert [order details] (orderid, productid, quantity, UnitPrice,
Discount)
                                values(@order_id, @prod_id, @quantity, @unitprice, 0)
--ITT AZ ÚJ RÉSZ:
        -- 3. raktárkészlet aktualizálása
                        declare @raktar_eredm varchar(50)
                        exec sp_raktárkészlet_aktualizalo @order_id, @raktar_eredm output
                        if @raktar_eredm = 'OK' begin
                                set @status = 0
                                set @status_message = cast(@order_id as varchar(20)) + ' sz.
rendelés sikeresen felvéve.'
                        end else begin
                                set @status = 3
                                set @status_message = 'HIBA: a raktárkészlet nem elegendő.'
                        end
                end
        end
        print 'Státusz: ' + cast(@status as varchar(50))
        print @status_message
        if @status = 0 commit tran else begin
                rollback tran
                print 'A tranzakció visszagörgetve.'
        end
end try
begin catch
        print 'EGYÉB HIBA: '+ ERROR_MESSAGE() + ' (' + cast(ERROR_NUMBER() as varchar(20)) +
')'
        rollback tran
        print 'A tranzakció visszagörgetve.'
end catch
go

--teszt
--beállítjuk a paramétereket
set nocount off
select * from Products where ProductName like 'Boston%'  --id: 40, unitsinstock: 900
update customers set balance=1000 where CustomerID='AROUT'
delete [Order Details] where OrderID in (select orderid from Orders where CustomerID='AROUT'
and EmployeeID is null)
delete Orders where CustomerID='AROUT' and EmployeeID is null
--most futtatjuk, kis mennyiségnél nem várunk hibát
exec uj_rendeles 'boston', 10, 'AROUT'  --nincs hiba
--ell.
select * from Customers where CustomerID='AROUT'
select top 3 * from Orders where CustomerID='arout' order by OrderDate desc
select * from Products where ProductName like 'Boston%'  --unitsinstock: 890 (OK)
--helyreállítás:
delete [Order Details] where OrderID in (select orderid from Orders where CustomerID='AROUT'
and EmployeeID is null)
delete Orders where CustomerID='AROUT' and EmployeeID is null
update Products set UnitsInStock=9 where ProductID=40 --kis készletet állítuk be
update customers set balance=1000 where CustomerID='AROUT'
--újra futtatjuk, a mennyiség miatt most hibát várunk
exec uj_rendeles 'boston', 10, 'AROUT'  --megjött a hiba
select top 3 * from Orders where CustomerID='arout' order by OrderDate desc --nincs új
rendelés
--az eredeti helyzet helyreállítása
```

```
delete [Order Details] where OrderID in (select orderid from Orders where CustomerID='AROUT'
and EmployeeID is null)
delete Orders where CustomerID='AROUT' and EmployeeID is null
update Products set UnitsInStock=900 where ProductID=40 --kis készletet állítunk be
update customers set balance=1000 where CustomerID='AROUT'
```

ÖNÁLLÓ FELADAT #20a: írjunk egy tárolt eljárást, amely egy terméknevet vár, és ha pontosan egy termék illeszkedik erre a névre, akkor az illető termék termék-kategóriáját visszaírja. Használjuk hozzá a 20. feladatban elkészített tárolt eljárást OUTPUT paraméterrel!

Beépített T-SQL függvények

A T-SQL beépített **skalár értékű** függvényei:

- Sztring-kezelő függvények, például LEFT, LEN, CHARINDEX, CONCAT, SUBSTRING, LTRIM, REPLACE, UPPER stb.
- Dátum-kezelő függvények, például GETDATE, DATEPART, DAY, MONTH, YEAR, DATEDIFF, DATEADD, ISDATE stb.
- Matematikai függvények, például ABS, ROUND, EXP, RAND, LOG stb.
- Konverziós függvények, például CAST, CONVERT, TRY_CAST, TRY_CONVERT, PARSE
- Logikai függvények: IIF, CHOOSE
- Analitikai és rang-függvények, melyek tipikusan több mint egy rekordot dolgoznak fel, például FIRST_VALUE, LEAD, ROW_NUMBER, RANK
- Egyéb rendszerfüggvények, például @@IDENTITY, @@TRANCOUNT, ERROR_NUMBER, NEWID, HOST_NAME stb.

A T-SQL beépített **aggregáló** függvényei egy értékhalmaz alapján egyetlen értéket adnak vissza, a NULL értékek figyelembe nem vételével (kivéve a COUNT függvényt). Az aggregáló függvényeket OVER nélkül csak a SELECT listában és a HAVING után lehet meghívni, GROUP BY használata esetén az alcsoportokra működnek. A legfontosabb aggregáló függvények a SUM, MIN, MAX, COUNT, AVG, VAR, COUNT, STDEV.

Felhasználó által definiált függvények

A felhasználó által definiált (röviden: felhasználói) függvények a tárolt eljárásokkal és triggerekkel együtt alkalmasak az üzleti logika megvalósítására. Fő jellegzetességeik:

- Deklarálás: a CREATE FUNCTION kulcsszóval. A paraméterlistát követően RETURNS után meg kell adni a visszaadott érték típusát. A függvény törzsét a BEGIN…END határolja. Az utolsó utasítás egy RETURN kell, hogy legyen.
- Visszatérési érték: egyetlen visszaadott érték, amely lehet skalár érték vagy tábla
- Tartalom: tartalmazhat belül deklarált változókat, lokális kurzorokat, vezérlési szerkezeteket, más függvények hívását, de *nem tartalmazhat olyan kódot, mely módosítja az adatbázis tartalmát*, vagy bármilyen mellékhatása van:
    - UPDATE, INSERT, DELETE utasításokat
    - Nem tartalmazhatja olyan függvények hívását, melyek nem determinisztikusak, például RAND, GETDATE.

- A felhasználó által készített tárolt eljárások sem hívhatók.
- Hibakezelés (TRY/CATCH illetve RAISERROR) használata sem megengedett függvényekben

- Meghívhatóság: a felhasználói függvényeket a T-SQL beépített függvényeihez hasonlóan a T-SQL utasításokba ágyazottan lehet meghívni. Minden esetben szükséges a függvényt tartalmazó *séma* neve is a meghíváshoz. Ha a függvényt rendszergazda jogosultságú felhasználó hozta létre, akkor ez a 'dbo', ha nem, akkor az egyéb létrehozott objektumokhoz hasonlóan a séma neve a login név.

PÉLDA: skalár értékű felhasználói függvényre, a dátum formázása:

```
go
drop function fn_ev_honap
go
create function fn_ev_honap (@datum datetime)
returns varchar(50) as
begin
	declare @eredm varchar(50)
	set @eredm = case
		when month(@datum) < 10 then cast(year(@datum) as varchar(4)) +'_0'+
cast(month(@datum) as varchar(2))
		when month(@datum) >= 10 then cast(year(@datum) as varchar(4)) +'_'+
cast(month(@datum) as varchar(2))
		else 'N.A'
	end
	return @eredm
end
go
--SELECT lekérdezésben felhasználható:
select e.employeeid, lastname, dbo.fn_ev_honap(orderdate) as honap,
count(orderid) as rend_szam
from employees e left outer join orders o on e.employeeid=o.employeeid
group by e.employeeid, lastname, dbo.fn_ev_honap(orderdate)
order by lastname, honap
```

Az üzleti logika megvalósítása könnyebben elvégezhető tárolt eljárásokban, ha a gyakori feladatokra felhasználói függvényeket használunk.

ÖNÁLLÓ FELADAT #21: írjunk függvényt, amely a paraméterül kapott alkalmazott ID (EmployeeID) alapján visszaadja az illető eddigi rendeléseinek a számát! Teszteljük a függvényt SELECT lekérdezéssel!

ÖNÁLLÓ FELADAT #22 (szorgalmi): írjunk függvényt, amely a paraméterül kapott rendelés ID (OrderID) alapján 1 értéket ad vissza, ha az illető rendelés minden tétele kielégíthető a raktárból, ha nem, vagy pedig a rendelés nem található, akkor pedig 0–t! Teszteljük a függvényt SELECT lekérdezéssel!

ÖNÁLLÓ FELADAT# 23: írjunk függvényt, amely két, datetime típusú változó közül visszaadja a korábbit, ha valamelyik NULL, akkor pedig a 'N.A.' stringet!

A **tábla-értékű függvények** szintaxisa annyiban különbözik, hogy típusnak a TABLE kulcsszót kell megadni. Az egyszerűbb esetben a függvény egyetlen SELECT utasítást tartalmaz, ekkor nem kell a tábla szerkezetét előre megadni. A tábla értékű függvényt a közönséges táblák helyén (például FROM után) lehet felhasználni.

PÉLDA: a függvény visszaadja a paraméterül kapott termékkategória termékeinek a nevét és az illető termék rendeléseinek a számát.

```
create function kateg_termekei (@kat_id int) returns table as
return (
```

```
        select p.ProductName as termeknev, COUNT(*) as rendelesek_szama
        from Products p inner join [Order Details] od on p.ProductID=od.ProductID
        where p.CategoryID=@kat_id
        group by p.ProductID, p.ProductName)
go
select * from dbo.kateg_termekei(1) where rendelesek_szama>30
```

ÖNÁLLÓ FELADAT #24: írjunk függvényt, amely egy táblában visszaadja a paraméterként kapott azonosítójú alkalmazotthoz tartozó területek nevét és régióját (RegionDescription és TerritoryDescription mezők)!

Triggerek

A triggerek olyan speciális tárolt eljárások, melyeket valamely előre megadott feltétel bekövetkeztekor a rendszer automatikusan hajt végre, ezáltal a passzív adatbázis reagálóvá, aktívvá válik. A leggyakrabban alkalmazott trigger-típus a DML-trigger, mely egy táblához van csatolva, és valamely adatmódosító utasítás (UPDATE, DELETE, INSERT) hatására fut le, mégpedig default esetben a kiváltó utasítás *után*[11]. A trigger meghívhat más tárolt eljárásokat és függvényeket, illetve az általa végzett műveletek kiválthatják más triggerek, sőt rekurzívan *saját maga* működését is. Egy táblán több, akár azonos típusú trigger is lehet, ebben az esetben a végrehajtásuk sorrendjét különféle szabályok alapján állapítják meg. Hibakeresés céljára a triggerek tartalmazhatnak PRINT utasításokat, azonban visszatérési értékük nincs, ahogy nincsenek bemeneti paramétereik sem. *Ha egy triggerben hiba történik, akkor a triggert kiváltó SQL utasítás is visszagördül.* A triggerek tipikus alkalmazásai:

- Olyan adatbázis-integritási szabályok definiálása és érvényben tartása, melyek több mezőt vagy táblát érintenek, ezért nem oldhatók meg egy CHECK kényszerrel.
- Automatizált üzleti folyamatok megvalósításának a támogatása.
- Háttér-adminisztráció, például tábla-módosítások naplózása.

Az SQL Serveren a trigger futás közben két speciális virtuális táblában fér hozzá a működését kiváltó utasítás által módosított adatokhoz:

- „**inserted**” nevű tábla: az új rekordok, INSERT és UPDATE triggerek esetén. Ne feledjük, hogy egyetlen UPDATE, DELETE, INSERT utasítás több rekordot is módosíthat!
- „**deleted**” nevű tábla: a régi rekordok, DELETE és UPDATE triggerek esetén

A triggert a CREATE TRIGGER utasítással lehet létrehozni, mely után meg kell adnunk a tábla nevét és a trigger típusát.

PÉLDA: minden új rendelés beszúrásakor emeljük meg az alkalmazott fizetését 2%-kal

```
drop trigger tr_uj_rend
go
create trigger tr_uj_rend on orders after insert as
begin
        declare @i int
        select @i=COUNT(*) from inserted
        print 'Beszúrt rekordok száma: '+cast(@i as varchar(50))
        update Employees set Salary=Salary*1.02
               where EmployeeID in (select EmployeeID from inserted)
end     --több rekordra is működik
go
```

[11] A másik lehetőség az, hogy a trigger a DML utasítás *helyett* fut le.

```
--teszt
select salary from Employees where EmployeeID=3 --100
insert Orders (EmployeeID) values (3)
select salary from Employees where EmployeeID=3 --102
```

ÖNÁLLÓ FELADAT #25: egy termék rekord módosításakor másoljuk ki egy products_log nevű táblába a módosult termék azonosítóját, nevét és raktárkészletét!

PÉLDA: Egy mindkét virtuális táblát használó triggerrel a már sokféleképpen megoldott raktárkészlet-aktualizálás is elvégezhető:

```
drop trigger tr_demo
go
create trigger tr_demo on [order details] after update as
declare @ord_no int
begin try
	select @ord_no = count(*) from inserted
	print 'updating records: ' + cast(@ord_no as varchar(50))
	update Products set UnitsInStock = UnitsInStock-(i.quantity-d.quantity)
	from products p inner join inserted i on p.ProductID=i.ProductID
		inner join deleted d on i.ProductID=d.ProductID
end try
begin catch
	print 'Hiba: túl nagy mennyiség a rendelési tételen.'
end catch
go

--teszt
select top 1 * from [Order Details] --quantity 12
update [Order Details] set Quantity=13000 where OrderID=10248 and ProductID=11  --hibát
váltunk ki a túl nagy mennyiséggel
select top 1 * from [Order Details] --quantity 12 (az update visszagördül magától)
```

ÖNÁLLÓ FELADAT #26: a logika értelemszerű módosításával írjuk át a fenti triggert úgy, hogy új rendelés beszúrásakor, tehát INSERT esetén működjön!

Bővebben a T-SQL programozásról:

- T-SQL reference: https://technet.microsoft.com/en-us/library/bb510741(v=sql.110).aspx